Lifelong Learning for a Kresling-Origami Robot:

A Plug-and-play Module for Supervised Learning of Target Adaptations

E. De Vroey

***Abstract*—Soft robots offer a variety of useful applications. However, the nature of their design makes them challenging to control using traditional techniques. Many applications therefore rely on machine learning-based methods, learning "blackbox" control policies or dynamic models of the robot. This often results in overly complex and opaque solutions for the problem at hand.**

In this paper, a simplified approach is considered: Using a common feedforward neural network as a *target adapter* module, learned adaptations are added to the desired targets that are fed to a simple PID controller, essentially *deceiving* it into a better performance. This adapter is completely separate from the controller itself, allowing for relatively simple controllers to control–and adapt to–complex and evolving systems.

The approach is tested in simulation on a simple mass-spring-damper control problem, where qualitative results show near-immediate improvement in controller performance. The approach is then tested on a demonstrator origami robot, which relies for its functionality on the dynamics of the Kresling origami spring. Results show that this simple adaptation approach is robust to both changes in controller tuning and changes in system dynamics. However, the method is sensitive to the sampling frequency at which training data is recorded.

I. Introduction

Cooler figure (maybe some sort of collage that somehow encompasses the entire process?), caption

Lifelong learning, the ability of a system to continuously adapt to new information throughout its lifetime, is a much sought-after feature for many applications. Rather than relying on policies that are *set in stone*, these systems learn and evolve autonomously, changing and improving their behaviors as they encounter new tasks or environments. This ability makes lifelong learning a powerful tool for systems that operate in uncertain or evolving scenarios, where changes are common or to be expected. Not in the least for robotics.

Robots, like all mechanical systems, wear down over time—joints become more compliant, motors weaken, and sensors lose precision. Traditionally, maintaining a robot's performance might require manual software updates or recalibrations every so often. But as robots become more complex, especially in fields like soft robotics, where flexible, compliant materials introduce additional challenges, the need for autonomous adaptation becomes clear: The materials used in the construction of soft robots are by design less sturdy and thus more prone to much faster deterioration, which makes manual updates or recalibrations impractical.

In addition, the complex system dynamics introduced by the unconventional robot configurations and use of materials make modeling their dynamics and control a difficult task. Deriving an analytical model for the dynamics of a soft robot can become so complicated to the point that it is impractical.

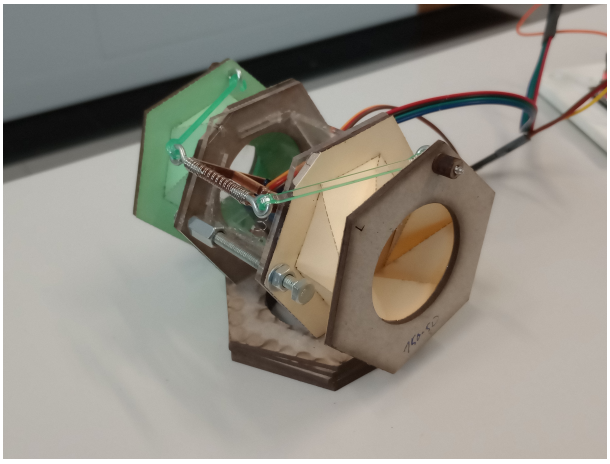


Fig. 1

Soft-robotics applications therefore heavily rely on the use of machine learning (ML) methods Bhagat et al., 2019; Kim et al., 2021. These "blackbox" techniques offer the possibility of modeling the robot's dynamics and allow for continuous adaptation but are often very opaque or complex, even though the task or environment setting of the soft robot could be something fairly simple.

This paper proposes a method of enhancing a traditional controller with ML abilities. The idea is that a basic controller can be tuned to a bare minimum sufficient performance, without worrying about the variations in dynamics introduced by soft robots. This controller is expanded with a separate module that takes care of the learning and adapting part of the challenge. The module then only needs to improve the performance of the controller, without requiring knowledge on the dynamics of the system, nor is the controller completely replaced by an uninterpretable "blackbox" policy. The controller works in cooperation with the ML-module, separating the interpretable basic control from the opaque learning and adapting.

Techniques like parameter adaptation work much in the same way. An adapting or tuning module, quite often ML-based, can be "plugged" into a base controller, tuning its parameters *on the go* to achieve an optimal performance (Annaswamy & Fradkov, 2021). However, parameter adaptation requires knowledge on the architecture of the controller (e.g., how many, and which parameters should be tuned), and such a module must thus be designed separately for each controller.

The proposed method tries to mitigate this downside by offering a "plug-and-play" module that can improve a con-

troller's performance by simply adapting the desired target it receives. In this way, no previous knowledge on the type or tuning of the controller is required. In other words, as long as the fundamental assumptions of a controller are satisfied—i.e. it requires a state and a target as input and yields a control action—the method can remain completely agnostic to the specific controller being used.

With soft robots offering many advantages in miniaturization (Tang & Wei, 2022), the physical space for controllers with heavy computational power is limited. The "plug-and-play" module offers the advantage of externalizing the heavy computational task from the soft robot, which allows the (miniaturized) robot to just run a basic controller with low computational capabilities.

As a demonstration of the technique, this target adaptation approach is tested on a demonstrator origami robot, constructed in part from paper and which relies on the widespread Kresling pattern (Kresling, 2008) for its design. The Kresling origami spring (KOS) is characterized by its non-linear stiffness with deflection and bistable behavior (Masana & Daqaq, 2019), making it an interesting building block for many soft-robotics applications (Masana et al., 2024). The demonstrator robot is thus a good representation of a large cross-section in soft robot designs, as well as a useful playground for testing the adaptability of the method to more complex dynamics.

II. Related Work

A. Soft Robotics

Soft robotics is a field that has gained much interest over the past decade. Soft robots are constructed from deformable, soft materials which offer advantages in fields such as human-robot interaction, where their inherent compliance makes them safe to interact with. Other use-cases range from manipulating fragile objects, to medical treatment delivery inside the human body, and search-and-rescue (Hawkes et al., 2017; Kim et al., 2021; Ze et al., 2022).

Origami robots are a subset of soft robots. Inspired by the Japanese art of paper-folding, these robots rely on creases in their materials for both structural integrity and actuation methods. This allows for highly miniaturized designs that can self-deploy and can be actuated using affordable, lightweight mechanisms (Tang & Wei, 2022).

Of course, although the materials used often bend rather than break, they *are* prone to deteriorate faster than their rigid counterparts. Hence, combined with the aforementioned difficulty of modeling the dynamics, the usefulness of lifelong learning in this field becomes obvious. Additionally, the rapid deterioration makes soft robots a fast and economical playground for testing lifelong learning techniques as well.

B. Lifelong Learning

Many approaches to lifelong learning can be found in literature, with machine learning being an integral part in all of them. Broadly speaking, the approaches can be divided into four categories, which are not necessarily mutually exclusive: Methods that result in a "blackbox" control policy, often obtained through reinforcement learning, where a policy is found by trying to optimize a *reward* function (Bhagat et al., 2019). Methods that model the dynamics of the system, to be used in combination with other controllers or to optimize an ML-based policy (e.g. Gillespie et al. (2018) and Thuruthel et al. (2019), respectively). Special architectures, such as parameter adaptation or estimation mechanism, and iterative learning control, where a separate module capable of learning is combined with a base controller (Annaswamy & Fradkov, 2021; R. J. Li & Han, 2005). Finally, the use of mechanisms that invoke some kind of memory, either by modifying the update method to be less *destructive* (e.g. *learning without forgetting*, Z. Li and Hoiem, 2018), or by implementing some form of library that can be accessed to store and retrieve information or parameters, as in e.g. Wang et al. (2020).

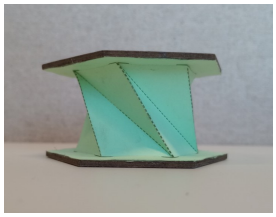


Fig. 2: Fully constructed 6-sided KOS.

The techniques most similar to the method discussed in this paper are parameter adaptation/estimation and iterative learning control (ILC). In parameter adaptation, ML-based methods are used to directly tune the internal parameters of a controller. The various techniques used to achieve this rely on observing the interactions of the controller with the system or having access to the feedback signal of the controller (Annaswamy & Fradkov, 2021). Furthermore, the estimation mechanism must be designed specifically for the parameters of the controller in question. Iterative learning control on the other hand only uses the feedback signal to improve a controller's performance. It achieves this by changing either the controller's reference signal (i.e. the target) or its control action, based on the observed error during a single trial (R. J. Li & Han, 2005). By iteratively applying this approach, the controller's performance can be improved. However, ILC is limited to systems that are highly repetitive; i.e. the same task is repeated many times.

C. Kresling Origami

The Kresling origami pattern is folding-pattern (Figure 3, *top-right*) used to create deployable cylindrical structures, also known as bellows (Figure 2). The pattern is based on the creases that naturally occur when twist-buckling a cylindrical structure (Kresling, 2008).

Due to the strain induced on the folds when twisting or compressing the structure formed by the pattern, it forms a torsional and translational spring with nonlinear stiffness. Depending on the specific design parameters, the spring can be designed to be either mono- or bistable (Masana & Daqaq, 2019): Monostable M1, M2 and M3 type springs differ in their stable deployment height being zero, non-zero and non-zero, respectively, and their stable potential energy being non-zero, zero and non-zero, respectively. Bistable types B1 and

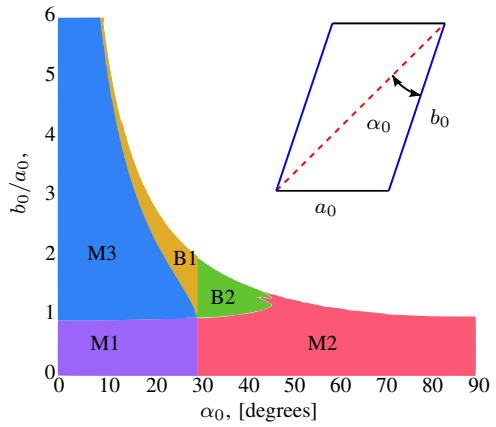


Fig. 3: Design map for a KOS with 6 sides reproduced from Masana and Daqaq (2019) and the corresponding parameters from the folding pattern, with mountain creases in blue and valley creases in dashed red.

B2 differ in their contracted stable height being non-zero and zero, respectively, and their respective potential energy being zero and non-zero. With the help of only three parameters, the design map shown in Figure 3 can be created from which a pattern can be designed that conforms to any of these categories.

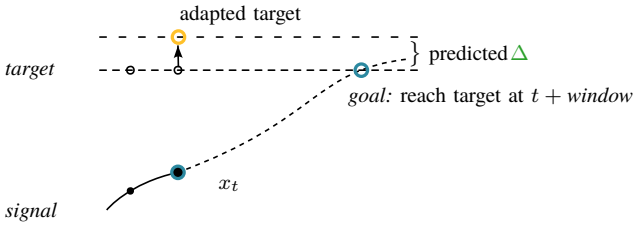
The bistable behavior of the KOS is a feature that has made the pattern popular in applications varying from mechanical switches, to aerospace structures and soft robots (Masana et al., 2024). The demonstrator robotic crawler in this paper is inspired by similar robots such as from Pagano et al. (2017) and Ze et al. (2022).

III. Methodology

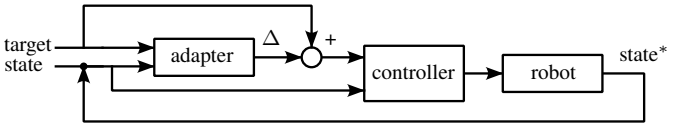
A. Target Adaptations

The method proposed in this paper aims to provide a way of improving a given controller's performance as well as fortifying it against evolving system dynamics *without* having access to the inner workings of the controller itself. In addition, the aim is not to replace the controller entirely but to introduce an additional module to the control scheme that enables lifelong robustness to evolving system dynamics. The advantage of such a method is that it could be applied to any system as a *plug-and-play* module.

The manner in which this will be achieved is by *target adaptation*. An *adapter* module modifies the target used by the controller to improve its performance. Consider a scenario where a controller yields a control action u to reach a certain desired target state $\mathbf{x}_d$ within a certain time window. As the system dynamics evolve, suppose this action u is now insufficient to reach the target. In response, based on the state of the system and the desired target, the adapter module *lies* to the controller about reaching $\mathbf{x}_d$ and instead provides it with $\mathbf{x}_d + \Delta$. For this adapted target, the controller now yields a larger action u^* that *does* result in the system reaching state $\mathbf{x}_d$. Figure 4a illustrates the desired outcome of the target adaptation method, the basic control scheme is presented in Figure 4b.



(a) By supplying the model with the controller's target position and current state x_t it yields a Δ that can be added to the current target, which will supposedly result in the controller reaching the target within the specified window.



(b) The target adaptation control scheme: The adapter is used to offset the given target before presenting it to the controller, essentially *lying* to the controller in order to improve its performance.

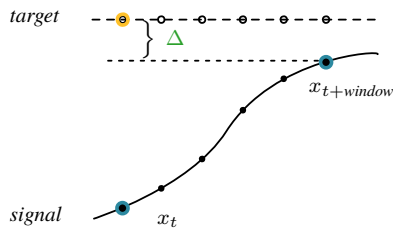
Fig. 4: Target adaptation.

At a glance, this method resembles that of iterative learning control (ILC). However, in ILC, the adaptation to the reference signal is directly related to the error signal, resulting in effective behavior for a very constricted task setting, where the same task is repeated over and over. Here, the goal is to have a more universal improvement in performance, and to detach the method from the error signal.

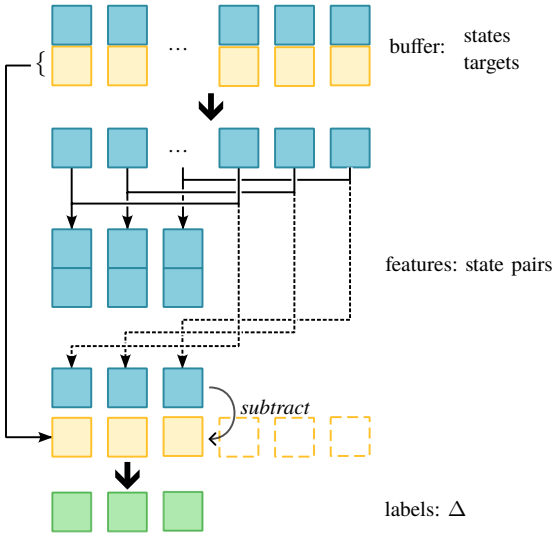
To achieve this, the adapter is implemented as a feedforward neural network with to hidden layers with non-linear activations. To train this model to learn suitable target adaptations, interactions of the controller with the system are recorded. Specifically, a buffer stores the recorded system states with corresponding targets. The model is then provided with pairs of system states that are obtained from the buffer and are spaced by a certain time window $\mathbf{x}_t$ and $\mathbf{x}_{t+\text{window}}$. From these pairs, the model infers the target it 'believes' the controller aims to reach. However, instead of predicting the target directly, the model learns the difference $\mathbf{\Delta}$ between the target $x_{d,\,t}$ and the latter state in the pair (Figures 5a and 5b).

The model can be trained in a supervised manner and then deployed. During deployment, it is presented with the current state $\mathbf{x}_t$ but cannot be presented with $\mathbf{x}_{t+window}$ (since this state is not known yet). Instead, the second state in the pair is set to the desired target. To the model, this represents a situation in which the desired target was reached from the current state within the specified time window. The idea is now that the trained model will produce a $\mathbf{\Delta}$ that–when added to the latter state in the pair (i.e., the desired target)–results in an adapted target for which the controller will produce a control action (as previously illustrated by Figure 4a). In short, given a current state $\mathbf{x}_t$ and a future state $\mathbf{x}_{t+window}$, the model learned to produce the desired target. Thus, when presented with $\mathbf{x}_t$ and $\mathbf{x}_{t+window} = \mathbf{x}_d$ (supposedly reached within the time window) it provides the (adapted) target that would "push" the controller toward that result.

This approach is thus completely detached from the error



(a) The target adaptation problem: Based on the current position x_t and the position some prediction window later $x_{t+window}$, the model learns to predict the target the controller is trying to reach by representing it as a Δ to be added to $x_{t+window}$.



(b) The feature and label extraction process: recorded states spaced by a specified prediction window are paired up and provided as features to the adapter, which learns to predict a $\mathbf{\Delta}$, being the difference between the target $\mathbf{x}_{d,\,t}$ and state $\mathbf{x}_{t+window}$.

Fig. 5: Target adaptation during the training phase

signal, and allows for non-repetitive settings, as opposed to ILC. ILC, however, also provides an approach of adapting the controller's action u directly. Trying to mimic this approach similarly to the target adaptation method described, one could train an adapter to predict a control action, or a series of actions, based on a state pair. The downside to this approach is merely a matter of personal opinion: as the adapter improves, it would essentially replace the controller (during deployment, it receives the state and target, and produces an action), therefore, the target adaptation was considered the more *elegant* approach.

Finally, the reason for predicting the difference $\mathbf{\Delta}$ between the state $\mathbf{x}_{t+window}$ and the desired target, rather than a direct prediction of the target, should also be clarified now: By designing the target adapter's output to be the difference $\mathbf{\Delta}$, the model can be initialized with parameters all close to zero (essentially leaving the desired target unchanged) and then steadily improve online as interactions of the controller and system are recorded, without the need for a pretraining phase.

IV. Illustrative Problem

To explore the proposed method, an illustrative problem is considered first. The problem consists of a simulation of

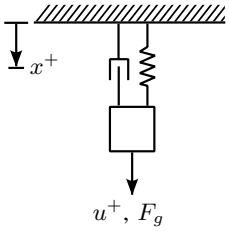


Fig. 6: Setup of the mass-spring-damper system in the toy problem with positive directions indicated.

a simple 1D mass-spring-damper system (Figure 6) and an imperfectly tuned PID controller. The mass is suspended in a hanging position with gravity acting on it continuously.

The system state $\mathbf{x}$ consists of the position and velocity x, $\dot{x}$, respectively. The controller aims to reach targets of the form $\mathbf{x}_d = [x_d,\, 0]$, but controls position only. At regular intervals, a random target position x_d is generated for the controller to reach and maintain. Each timestep, the adapter calculates the required Δ, and the controller computes the control action based on the current position and the adapted target position. The system's response to this control input is simulated and the tuple $(x_0, \dot{x}_0, u, x, \dot{x}, x_d + \Delta, 0)$ is added to the buffer. This buffer stores a limited number of tuples corresponding to a specified window of the most recent data. Once capacity is reached, the oldest element is removed at every timestep. At regular update intervals, the adapter is trained on the data in the buffer. It is important that the tuples in the training data always contain the target position that was actually provided to the controller, i.e., $x_d + \Delta$, and not the *true* desired target x_d. As the simulation progresses, the parameters of the dynamic system are gradually altered to simulate system degradation. The pseudo algorithm for the implementation can be found in Algorithm 1.

The simulation ran for 60s at 50Hz. The target was changed every 5s, and the adapter was trained fully online, updating every 15s. Training features were created for multiple prediction windows, this means the training features consisted of pairs $[\mathbf{x}_t, \mathbf{x}_{t+window}]$ for windows of 3, 5, 10, 15, and 25. The buffer's budget corresponded to the past 30s of data. As a baseline, a second controller and system were also simulated, identical to the described simulation, but without the addition of an adapter. To both the baseline and adapted signal, a small amount of Gaussian noise was added to simulate measurement noise.

A. A case where the controller is too aggressive

For a first simulation, a combination of a system and controller was chosen where the controller's gains were tuned too high. In Figure 7, the result of the simulation can be observed. At the start, and due to the near-zero initialization of the adapter's parameters, the signal response of the system controlled by the adapted controller is indistinguishable from that of the reference controller. However, as soon as the adapter has trained on the buffer, the changes to the target become visible.

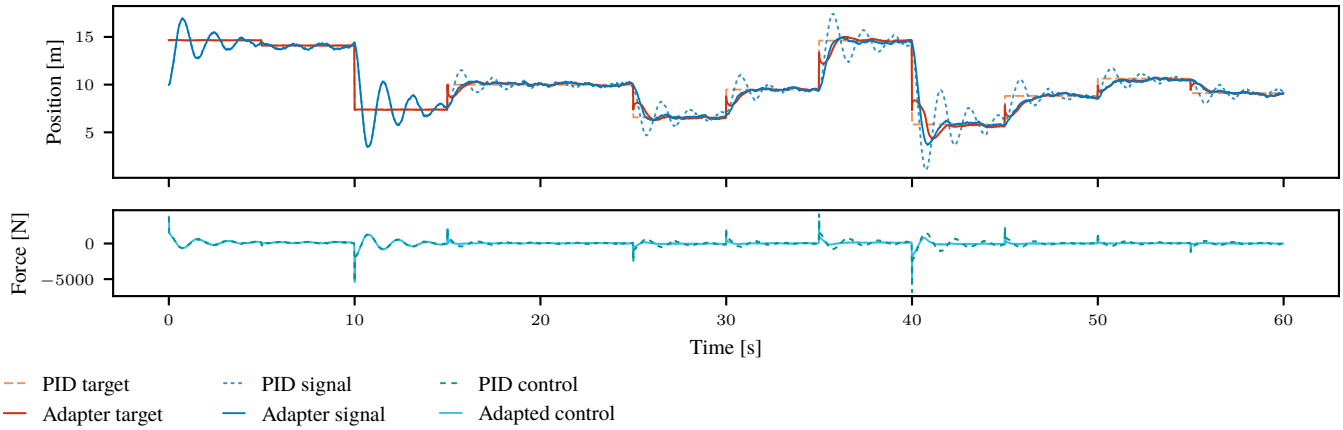


Fig. 7: The full minute simulation of the toy problem with an aggressive controller. *Top*: The system response x and target $x_d + \Delta$ of the adapted controller, alongside the response and target of the unadapted reference controller ($\Delta = 0$). *Bottom*: The resulting adapted control inputs from the controller receiving $x_d + \Delta$ alongside the unaltered reference control inputs. During the first 15s (3 targets) the adapter has not received updates yet, and the signal is thus identical to the reference signal.

Algorithm 1: Target adaptation

Data: system state $\mathbf{x}$, target $\mathbf{x}_d$, adaption Δ, control action u
Initialize model: *adapter*;
Initialize buffer: *buffer* $\leftarrow$ [];
for *each timestep* t_i **do**
  if *len(buffer) > threshold* **then**
    Remove the oldest element in the buffer;
  end
  if $t_i \mod target\_interval = 0$ **then**
    $\mathbf{x}_d \leftarrow random\_target()$;
  end
  if $t_i \mod update\_interval = 0$ **then**
    (features, labels) $\leftarrow$ *prep_data(buffer, window)*;
    adapter.fit(features, labels);
  end
  $\Delta \leftarrow adapter.predict([\mathbf{x}_0, \mathbf{x}_d])$;
  $u \leftarrow controller.compute\_control(\mathbf{x}_0, \mathbf{x}_d + [\Delta, 0])$;
  Simulate the system response $\mathbf{x}$;
  Update system parameters;
  Append $(\mathbf{x}_0, u, \mathbf{x}, \mathbf{x}_d + [\Delta, 0])$ to *buffer*;
  $\mathbf{x}_0 \leftarrow \mathbf{x}$;
end

The effects of the adaptation can be more clearly observed in Figure 8. Since the reference controller is too aggressive, the reference signal overshoots and oscillates significantly before settling. In contrast, the adapter counters this by bringing the target closer to the current state. This is the logical result of the buffer containing data corresponding to the overshoots, where the adapter would often encounter training features for which the labeled target position[1] x_d is somewhere in between x_t and $x_{t+window}$. Therefore, upon receiving the input feature

[1]Rather, the labeled $\Delta = x_d - x_{t+window}$, but the explanation provided is equivalent and somewhat clearer.

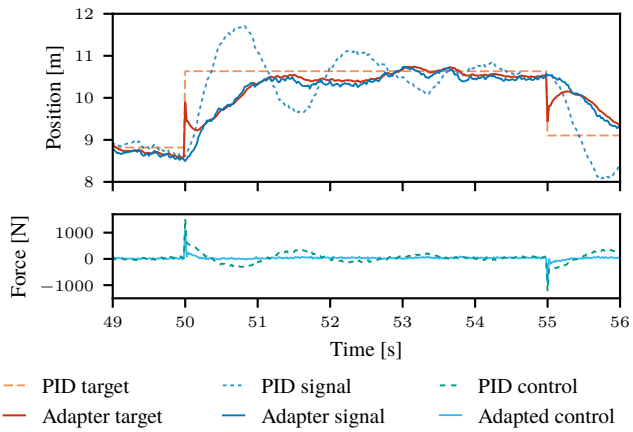


Fig. 8: A close-up of the second to last target in the one-minute simulation. The effects of the adapter can be more clearly observed: The overshoot (seen in the reference signal) is countered by bringing the target closer to the current state.

$[\mathbf{x}_t, \mathbf{x}_d]$ it produces and adapted target position in between x_t and x_d. As the state approaches the adapted target, the adapter starts producing targets closer to the reference target, guiding the system towards this desired state.

B. A case where the controller is under-responsive

For a second simulation, the system and controller were designed such that the controller was more *conservative* or *under-responsive*. Additionally, the signal produced by the reference controller has a clear steady-state error. The result of the simulation can be found in Figure 9.

Once more, the adapter is able to improve the controller's performance after just a single update. In Figure 10 it can be seen that the adapted target is adjusted away from the current state. This is because training features created from the data in the buffer frequently include state pairs $\mathbf{x}_t$, $\mathbf{x}_{t+window}$ where the target position x_d is positioned further from x_t than it

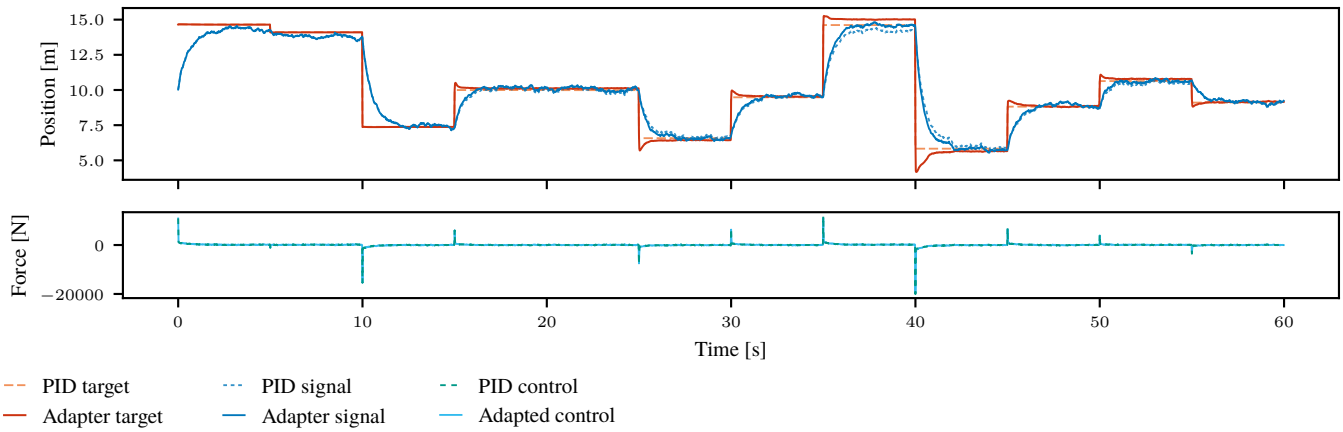


Fig. 9: The full minute simulation of the system with an under-responsive controller. *Top*: The system response and target of the adapted controller, alongside the response and target of the unadapted reference controller. *Bottom*: The resulting adapted control inputs from the controller receiving the altered target alongside the unaltered reference control inputs.

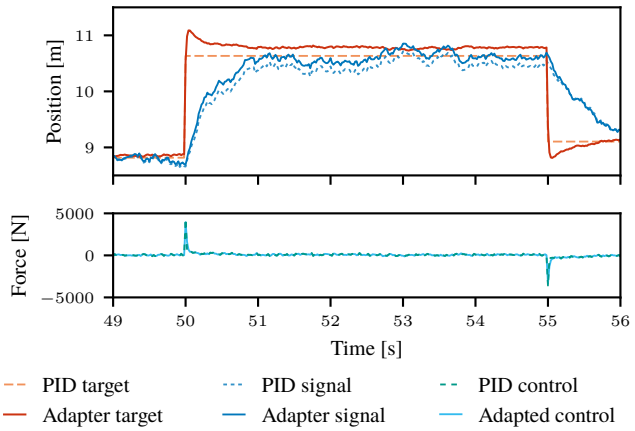


Fig. 10: A close-up of the second to last target in the one-minute simulation. Noteable observations: The target is initially moved further from the current state to 'pull' it towards the desired state in a shorter period. In addition, the adapted target is offset from the desired state as to counter the steady-state error seen in the reference signal.

is from $x_{t+window}$. As a result, the adapter learns to predict adaptations that would move x_d farther from x_t, making the controller more aggressive. Another effect of the adapter is the consistent offset applied to the target: In the training features, the adapter regularly encounters pairs $\mathbf{x}_t, \mathbf{x}_{t+window} = \mathbf{x}_t$, which appear to represent the target state being reached, even though $x_{t+window} \neq x_d$. These features are the result of the steady-state error present in the signal, leading the adapter to learn to apply an offset to the target to counteract this.

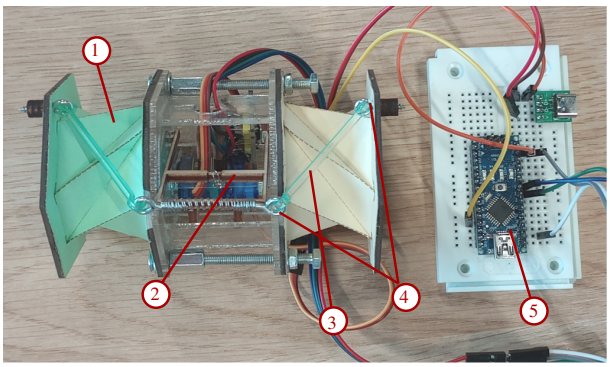
V. A Kresling Origami Robot

A. Mechanical Details

As mentioned in Section II-A, origami robots—and by extension, soft robots—are a suitable use-case for lifelong learning techniques. To evaluate the method described in Section III, a robot was designed that serves as a good analogue to the toy problem, whilst also using Kresling origami as a key part of its functionality. Figure 11 provides important nomenclature for reference.

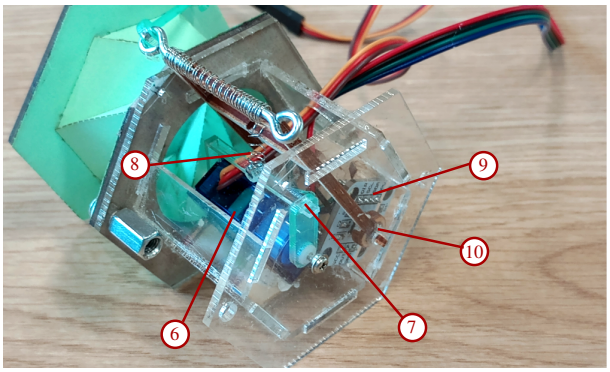
The robot is a simple crawler consisting of two Kresling origami springs (KOS) (Figure 11a, item 1) that are mirror images of each other, connected in series, with the actuation mechanism in between. The KOSs in question are of the bistable B1 type (Section II-C) with perforated folds to facilitate actuation (refer to Appendix D for design details and templates). Both springs can expand and contract in sync, producing a crawling-like motion when combined with specially designed "feet" (see Figure 12, *top*). To demonstrate target adaptation, however, the movement of the robot as a whole is not very important. Rather, the state of the KOSs is the state that will be controlled.

A KOS can be actuated both by linear motion (pushing or pulling on the spring) and by rotational motion (twisting). Since the two motions are directly related (i.e. twist-buckling, discussed in Section II-C), a contracting or expanding motion can be measured as the angle of the KOS' twist. The twist is induced by turning a rotating arm (2) which is connected to the ends of the KOSs by two links (3) that hook into eye-screws (4). To turn the arm, a micro servo motor (6) is employed. In the toy problem, the controller yields a force to exert on the system in order to reach the target position. However, most micro servo's come with a near perfect position controller built-in, which cannot be accessed or changed. Therefore, to bridge the gap between the simulation and the real-world implementation, the position output of the servo is converted to a force output by the use of a small, metal spring (8). The servo thus does not directly act on the actuation arm, but rotates a servo arm (7) instead, which pulls the spring and thereby exerts a force on the actuation arm. As previously mentioned, the state of the KOS can be measured as the angle of twist, since it is directly related to its contraction. Likewise, assuming perfectly rigid links with negligible clearance between the hooks and eye-screws, the angle of twist is directly related to the angle of the actuation arm. In the full implementation, the



1) KOS
2) Actuation arm
3) Connecting link
4) Eye screw
5) Arduino

(a) The Kresling origami robot.



6) Servo motor
7) Servo arm
8) Metal spring
9) Angle sensor
10) Magnet

(b) Actuation mechanism.

Fig. 11: Annotated views of the robot and its most important actuation mechanisms

robot's state $\mathbf{x}_t$ will thus be measured as the actuation arm's angular position and velocity $[\theta, \dot{\theta}]$. This angle is measured using a contactless sensor (9) that can measure the on-axis rotation of a small magnet (10) attached to the actuation arm. The sensor relays this information to an Arduino Nano (5) board, which runs a PID controller that returns the control action, i.e., an angular position for the servo. Furthermore, there are mechanical limits for the rotation of both the actuation arm and servo arm, imposed by the edge of the container housing the actuation mechanism and the supports holding the servo motor, respectively (Figure 12, *right*). However, the design is such that the servo's mechanical limit corresponds to a θ below the actuation arm's limit.

B. Software Implementation

The software implementation consists of two main components: The nominal control of the robot, and the target adaptation.

The Arduino board is responsible for the nominal control, implemented as a simple PID controller. Additionally, it takes care of calibrating the measurements upon startup, i.e. (re)setting the actuation and servo arm to their minimum "home" position (Figure 12, *left*).

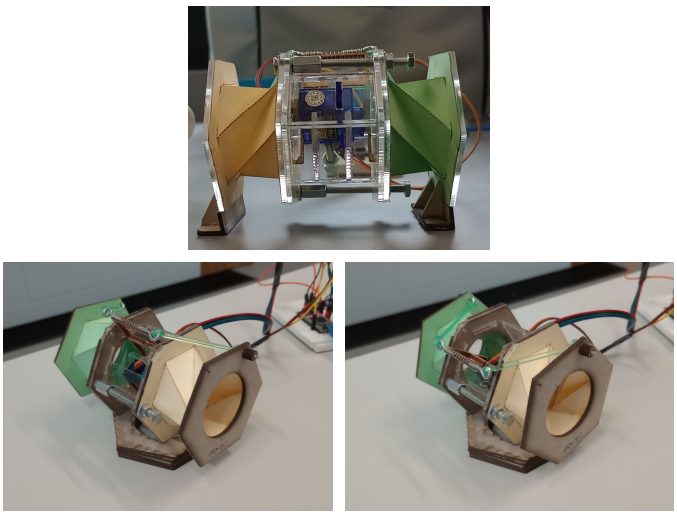


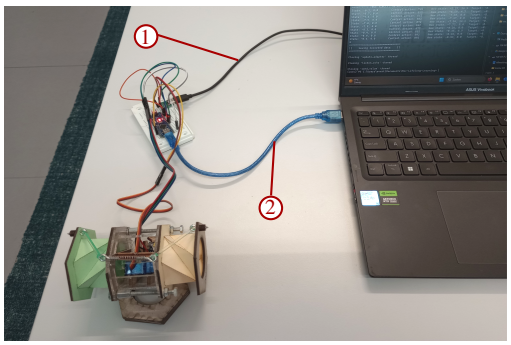
Fig. 12: *Top*: The "feet" of the robot grip when dragged backwards, but slide when pushed forward, allowing the robot to move forward, also known as *two-anchor crawling* (Calisti et al., 2017). *Left*: The robot at its fully extended state, corresponding to the "home" position of the servo and actuation arm. *Right*: The fully contracted robot, corresponding to the maximum (capped) servo output, near the mechanical limit of the actuation arm.

Target adaptation is implemented on an external computer. Through a serial connection (Figure 13, item 2) with the Arduino, it both receives the measurements to be stored in the buffer, and sends the adapted targets to be used by the PID controller. The main process was summarized in Section IV by Algorithm 1. However, on the computer, the several steps are divided amongst separate modules that act in parallel so that real-time performance is ensured.

A more detailed description of the software implementation can be found in Appendix A.

VI. Experimental Setup

For each experiment, a minimum of two sets of KOSs was required: One set serving as a baseline, with the Arduino receiving unadapted targets, and the second set used for testing target adaptation. All experiments used a 60s buffer and 15s intervals between updates, with the first update at 60s. The adapter trained on prediction windows of 3, 5, 10, 15, and 25 steps. The true target, with values depending on the experiment, was updated every 5s, following a predefined random sequence. This sequence amounts to 5min of deployment time and is repeated according to the full experiment duration. The signals corresponding to each 5min sequence are recorded and saved for evaluation. This way, the evolution of the performance metrics can be studied, and a fair comparison between the reference controller and adapted controller is ensured. Figure 13 shows the general experimental setup.



1) Servo motor power supply
2) Serial connection and Arduino power supply

Fig. 13: The test setup.

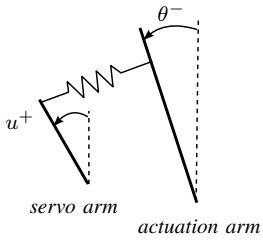


Fig. 14: Nominal signs of the angles for the servo and actuation arm during operation.

A. Sign Conventions and Unit Clarification

Before the experiments and results can be properly understood, it is necessary to clarify certain sign and unit conventions.

For example, contactless magnetic sensor denotes a clockwise rotation (as viewed from the top) w.r.t. the datum as a positive angle, and likewise, a counterclockwise rotation as a decrement in the angle (Rob Tillaart, 2024). In practice, this means that a positive control input from the servo (i.e. a positive angle) results in a decrement in θ. Because the actuation arm's calibration point is set when the servo is at its minimum output, the operational range of the actuation arm is entirely in the negative domain. Figure 14 illustrates the servo arm and actuation arm in a configuration representative of a typical scenario during the experiments, with the signs of the angles indicated.

In addition, The *Servo* Arduino library uses the `writeMicroseconds(value)` function to write to the servo. `writeMicroseconds(value)` maps a servo's range to a value between 1000 and 2000µs (depending on the servo's manufacturer), relating to the pulse width in microseconds of the signal that the Arduino sends to the servo motor (Arduino, 2024).

Finally, during the experiments, the Arduino initialized a *counter* value to include in the tuple sent to the computer. This counter starts at 0 and simply increments by 1 with each iteration of the Arduino's loop. Since the interactions between the Arduino and computer and the several threads on the computer are independent of timing (i.e., the modules do not have to "wait for each other"), the counter's value is essential to accurately plot the several signals recorded during the experiments.

B. Performance Metrics

The performance metrics used to evaluate the 5min sequences are the mean rise time, mean settling time, mean overshoot, mean absolute error, integrated absolute error, mean absolute velocity, normalized total variance, and mean control action.

The mean rise time is calculated as the mean of all the rise times corresponding to each target in the 5min sequence. The rise time is defined as the time it takes the robot to come within 10% of the target state, as calculated from the initial position.

Similarly, the settling time is the average of the settling times for each target. The robot state is considered settled when it remains within 5% of the target state, as calculated from the initial position.

The mean overshoot is the average of the maximum overshoots observed for each target.

The mean absolute error represents the average difference between the robot's angle θ and the true target θ_d.

The mean absolute velocity refers to the measurements of $\dot{\theta}$. This metric is meant to provide a measure for the "chaoticness" of the robot's signal response.

The normalized total variance is calculated as the sum of the absolute of all increments and decrements in the control action, normalized over time[2]. This aims to quantify the controller's "effort".

VII. Experiments and Results

The performance of the target adapter is evaluated over three types of experiments. In the first type, the PID controller's tuning is varied, while maintaining consistent KOS characteristics (Section VII-A). For the second group of experiments, the system dynamics are varied, namely the degree of perforation in the folds of the KOS, while keeping the PID controller identical across tests (Section VII-B). Finally, the performance is tested for a crawling gait to observe the effect of the method in a highly restricted setting (Section VII-C). For the comprehensive set of results, refer to Appendix B.

FOR ENTIRE SECTION: quantify the results: x% improvement of y, etc.

A. Varying the PID Controller

For the first experiment, the method's robustness to the use of different PID controllers is tested. The robot is equipped with the standard set of KOSs, using perforations of 1.5–0.5mm cut–skip ratio. During this experiment, the values of the targets in the random sequence are integer values ranging between -3 and -25 degrees. The robot has 5s to reach and settle around a target before it is provided with the next target in the sequence.

[2]Actually, it is normalized over the *count*, see Section VI-A.

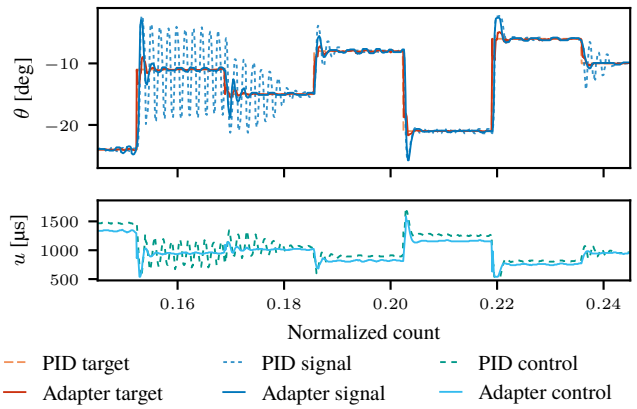


Fig. 15: Qualitative comparison between the behavior of the baseline controller and target adapted controller for the *slow-aggressive* PID controller.

1) A controller that is slow but aggressive: The first PID controller has a disproportionally high integral gain, resulting in an aggressive but slow to respond controller. This controller yields particularly interesting results in combination with the bistable nature of the KOS. Figure 15 shows the comparison between the signal responses from the baseline controller and adapted controller, illustrating the behavior of the controller.

During the full 2h duration of the experiment, each 5min sequence was recorded and evaluated using the metrics discussed in Section VI-B. The result can be seen in Figure 16.

Note the immediate (after the first 5min) improvement in overshoot and absolute error. The controller with target adapter initially has high rise times and low overshoots, but then evolves toward a stable behavior that retains short rise time, small position errors, and consistent overshoot values. As time progresses, the metric values for the unadaptive reference controller generally decrease, with the exception of the rise time. The adapted controller, however, seems to achieve this level of performance from the start.

Interpretation: The conjecture is that the potential barrier between the two stable states of the KOS slows down the controller's response, resulting in a higher control input. As the barrier is overcome, the KOS now complements the controller's action, resulting in the overshoot. Depending on the vicinity of the target to the KOS's potential barrier, the controller can enter an unstable state. As the potential barrier to transition between the two stable states of the KOS decreases, the reference controller has less difficulty maintaining a stable response. This explains the steady decrease in settling time, absolute velocity, and total variance, which reflect the decreasing irregularities in the signal and the controller's corrective actions. The target adapter effectively shifts the target closer to the system's current state, facilitating a smoother transition to the desired state with reduced overshoot, thereby minimizing the risk of the controller entering an unstable cycle.

2) An under-tuned controller: The second PID controller is tuned with low gains, resulting in slow and under-responsive behavior. Figure 17 displays the comparison between the baseline controller and the controller with target adapter.

The evolution of the metrics is shown in Figure 18.

The controller with adapter clearly has lower rise times and absolute errors than the reference. However, it also exhibits higher overshoots, whereas the reference controller even slightly undershoots toward the end of the 2h experiment. The adapted controller results in higher mean absolute velocity and total variance.

Interpretation: The signal in **??** shows a clear reason for the lower rise times, namely that the target is shifted to more extreme positions as to yield larger control inputs from the controller. This corresponds to the observed larger overshoot and higher mean absolute velocity. The adapter makes the controller more *aggressive*, as seen in the increased controller *effort* (total variance). However, the adapter also seems to introduce unnecessary adaptations once the steady state is reached, further increasing the mean absolute velocity. These small oscillations are insignificant compared to the overall improvement in the mean absolute error.

B. Varying the System Dynamics

For the second type of experiment, the PID controller is tuned to a highly responsive performance on a robot with KOS perforations of a 1.5–0.5 mm skip-to-cut ratio. The KOS perforations used are 0.25–0.25 mm, 0.5–0.5 mm, 1.5–0.5 mm, and 2–0.5 mm, representing various stages of KOS degradation. Once again, the values of the targets in the random sequence are integer values ranging between -3 and -25 degrees, and the robot has 5s to reach and settle around a target.

1) 0.25–0.25 & 0.5–0.5 Perforations: The 0.25–0.25mm and 0.5–0.5mm perforations represent KOSs in a state of lower degradation than the standard 1.5-0.5mm perforation. The evolution of the metrics over the course of the 1h experiment is shown in Figure 19, showing the average of the results from both experiments.

On average, the adapted controller has lower rise times, settling times, and absolute errors, but higher overshoot, absolute velocity, and total variance. Another observation is that the difference in the reference controller's performance between the two experiments drifts further apart over the full duration. However, the adapted controller's performance over the two experiments appears less varied. Overall, the relative difference in metrics between reference and adapted controller resembles that of the under-tuned PID experiment.

Interpretation: Since the perforations aim to represent a less degraded KOS, i.e. stiffer, the gains of the PID (tuned to a 1.5–0.5mm perforated KOS) are likely set too low. The situation is thus analogous to the under-tuned PID experiment, and this shows in the behaviour of the target adapter: The targets are shifted toward more extreme positions, resulting in the lower rise time.

2) 1.5–0.5 & 2–0.5 Perforations: The 2–0.5mm perforation aims to resemble a KOS that has been more deteriorated. The PID controller is initially tuned for the 1.5–0.5mm KOS. For the 1.5–0.5mm KOS experiment, with the runtime set to 2h, the aim is to assess how the controllers perform as the system gradually shifts into a more over-tuned condition.

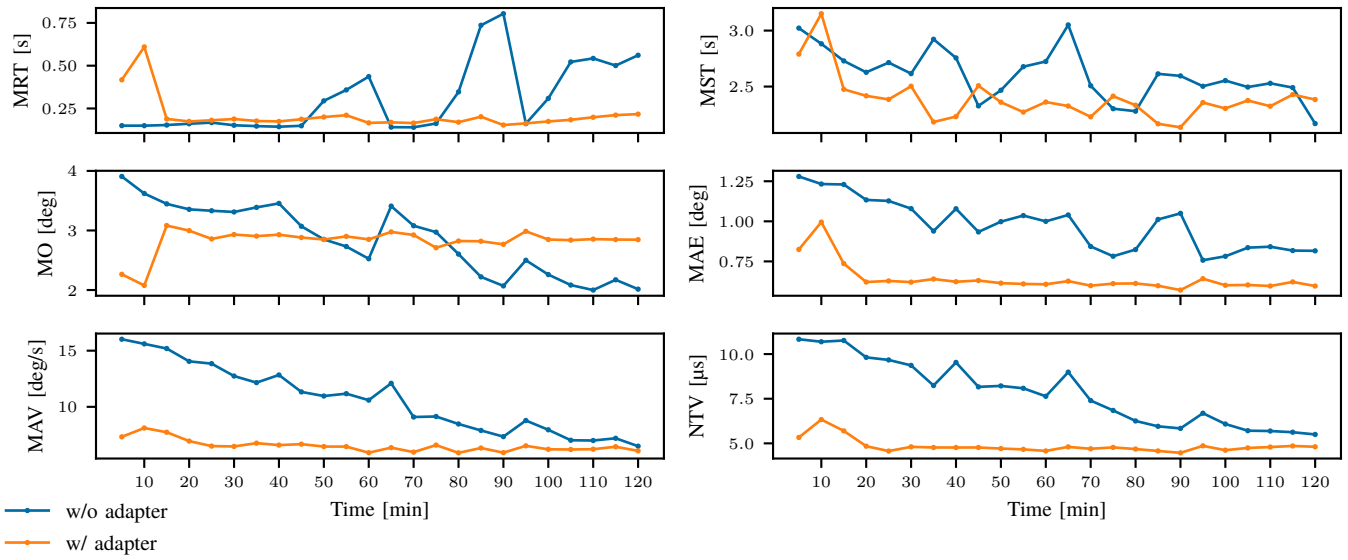


Fig. 16: Metrics evolution over the 2h duration of the experiment for an *aggressive* but *under-responsive* PID controller. Each point represents the calculated value of the metric on identical 5min sequences of targets. Plots for all metrics and numerical values can be found in Figure B.1 and Table B.I, respectively.

just MO, MAE, MAV, (and NTV,) 1 column

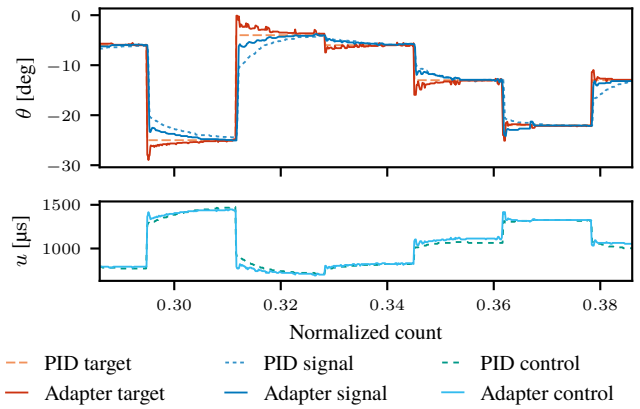


Fig. 17

Figure 20 shows that the general performance of the target adapter consists of higher settling times, overshoot, mean absolute velocity and total variance compared to the reference, especially as the experiments progress. In fact, the only metric where the target adapter metrics are smaller than those of the reference, is the rise time.

Inspecting Figure 21, it can be observed that by the end of the experiment, the target adaptations have become significantly noisy, resulting in chaotic behavior. The result of this is the increased settling time, mean absolute error, mean absolute velocity, and controller effort. However, as shown in Figure 20, the adapted controller initially demonstrates better performance concerning settling time and mean absolute error. Indeed, Figure 21b illustrates that, in the first 30min of the experiment, the target adapter produces minimal and relatively stable adaptations, leading to improved rise times and lower absolute error.

Interpretation: In the case of heavily deteriorated KOSs, the target adaptation seems to degrade the performance. The initially improved rise times can be attributed to the highly oscillatory nature of the signal response, rather than genuine performance gains. However, the fact that the target adapter shows improvement over the reference controller in the early stages of deterioration is promising. The highly responsive PID controller is quick to react, reducing the need for extreme adaptations as seen in the previous experiments. As the KOS further degrades, it sometimes enters an unstable cycle: much alike the ones seen in **??**, Section VII-A, but at a higher frequency. This results in training data that contains many contradicting features, which results in increased noise in the adapter's predictions. Noisy targets in turn produce noisy control actions and consequently noisy signal responses.

C. Crawling Gait

The final experiment emulates the mode of operation that the robotic crawler was designed for. The targets in the sequences alternate between two angles on opposite sides of the potential barrier, specifically -8 and -18 degrees, mimicking the contraction and expansion typical of a crawling motion. To introduce variability and avoid identical datapoints, a small normally distributed random offset is added to each alternating target. The goal of this experiment is to evaluate how a constrained operational setting affects the adapter's compensation for the overshooting effect caused by the potential barrier. The experiment uses a KOS with a perforation 1.5–0.5mm and the highly responsive PID, tuned to this perforation. Given the rapid degradation of the potential barrier in the early stages, the experiment only runs for 20min. A reference

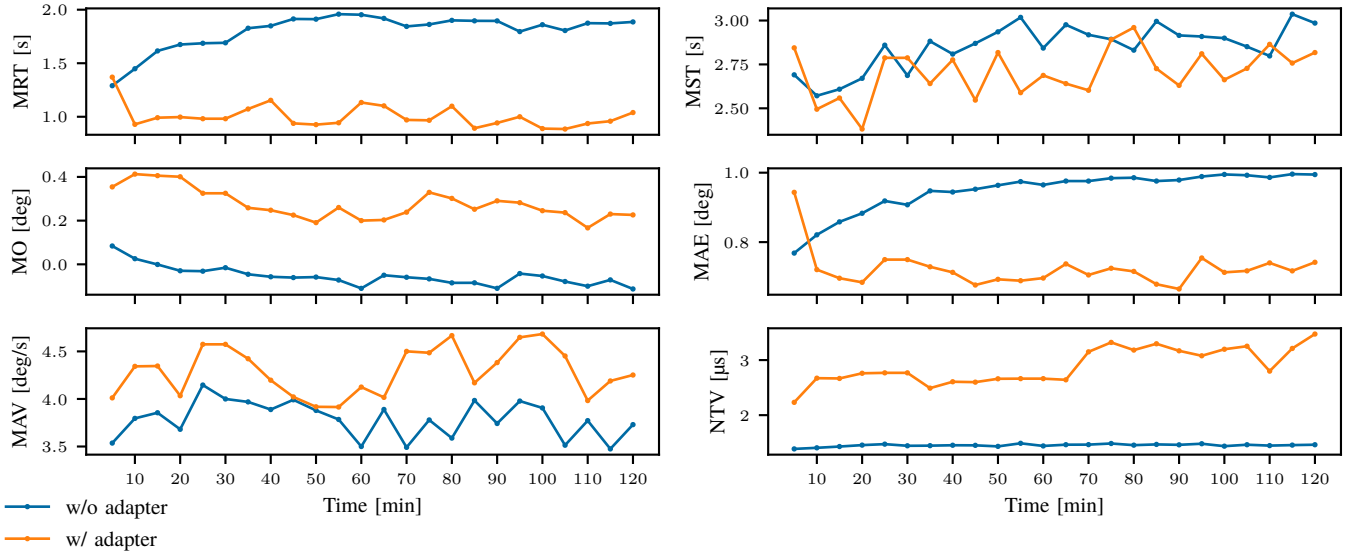


Fig. 18: Metrics evolution over the duration of the experiment for an *under-tuned* PID controller. Each point represents the calculated value of the metric on identical 5min sequences of targets.Plots for all metrics and numerical values can be found in Figure B.2 and Table B.II, respectively.

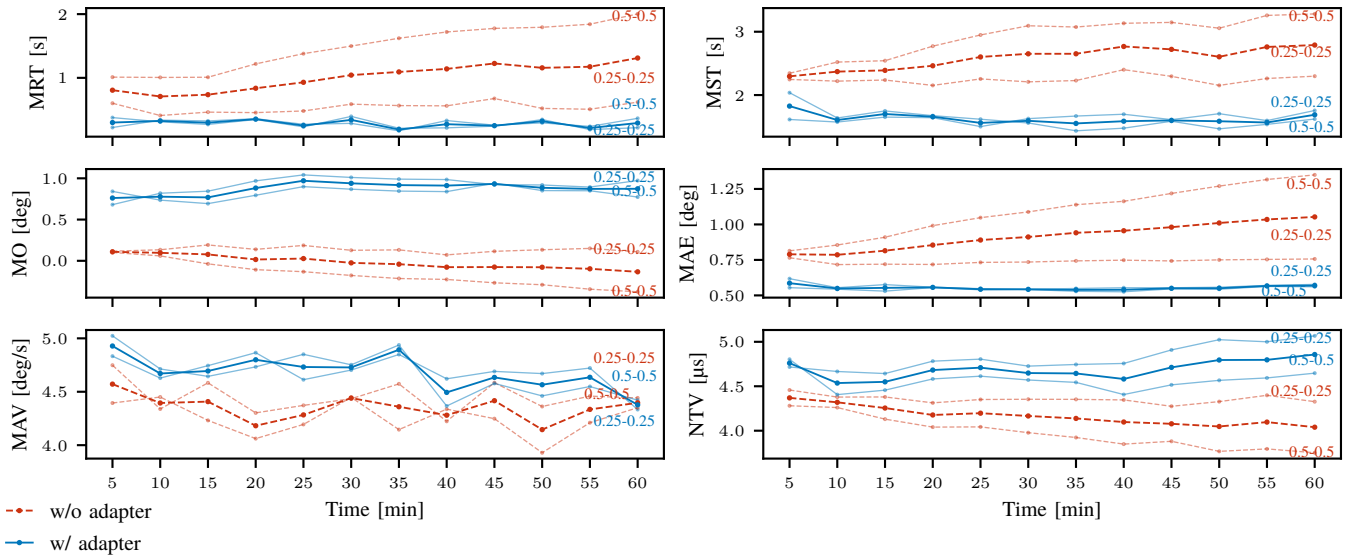


Fig. 19: Metrics evolution over the 1h duration of the experiment for KOSs in a lower state of degradation (0.25–0.25mm and 0.5–0.5mm perforations, see annotations), with their average overlayed. Each point represents the calculated value of the metric on identical 5min sequences of targets. Plots for all metrics and numerical values can be found in Figure B.3 and Table B.III, respectively.

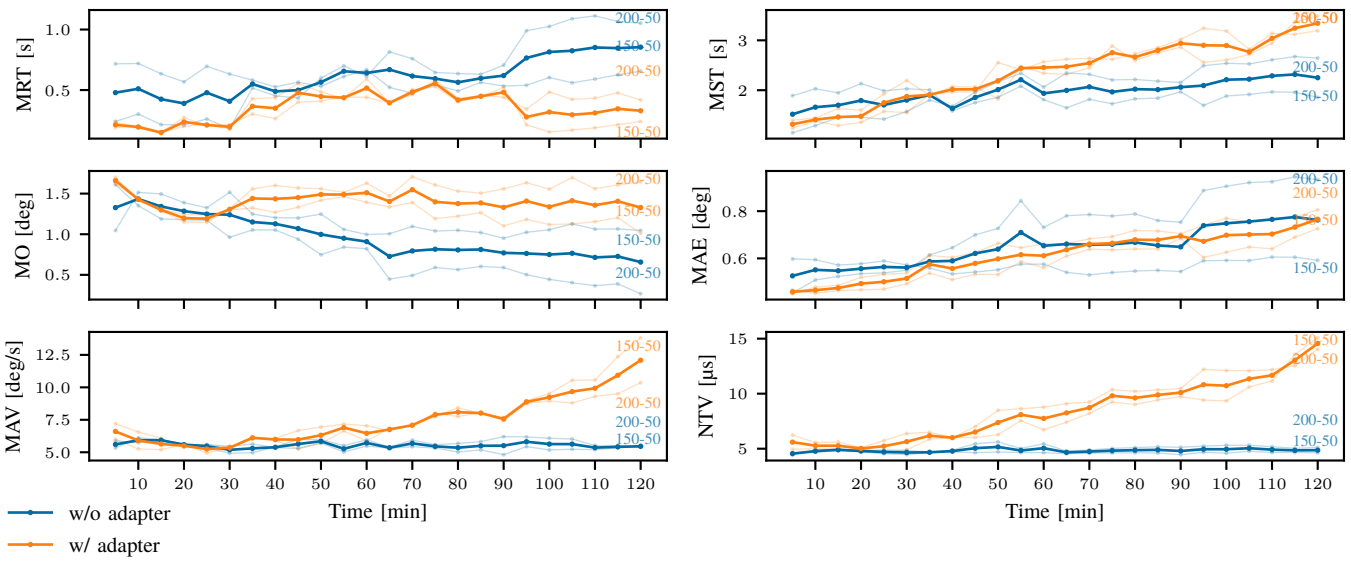


Fig. 20: Metrics evolution over the 2h duration of the experiment for KOSs in a higher state of degradation (1.5–0.5mm and 2–0.5mm perforations, see annotations), with their average overlayed. Each point represents the calculated value of the metric on identical 5min sequences of targets. Plots for all metrics and numerical values can be found in Figure B.4 and Table B.IV, respectively.

controller is compared with two target adapter configurations: one pretrained and then fine-tuned online, and one that is entirely trained online.

Figure 22 shows the evolution of the metrics for the three runs over the span of 20min. The results show consistently lower rise and settling times, and mean absolute error for both implementations of the target adapter compared to the reference controller. These improvements come at the cost of significantly higher overshoot.

Signal responses are shown in Figure 23, note that the adapted signal is only shown for the fully online trained adapter, but it is very similar to that of the pretrained adapter. The improvement in rise time is clearly observable, and shows that the improved settling time directly results from this, rather than the reference controller having an unstable signal. In fact, the signal from the reference resembles that of an under-tuned controller. Target adaptations appear to be minimal, being only visual in close-ups of the signal. This corresponds to the relatively low difference in total variance between reference and adapted controllers, observed in the metric evolution.

Interpretation: Although the adaptations made to the targets appear negligible for both the pretrained as the online trained adapter, the metrics and signal response show a definite improvement. However, the cause of how these minor adaptations result in the observed improvements is not immediately apparent: the reference controller shows signs of being under-tuned, but the adapter does not produce adaptations similar to those of an under-tuned controller in Section VII-A. On the other hand, the consistent improvements seen across both adapter configurations challenge the notion that these improvements are merely coincidental.

VIII. Discussion

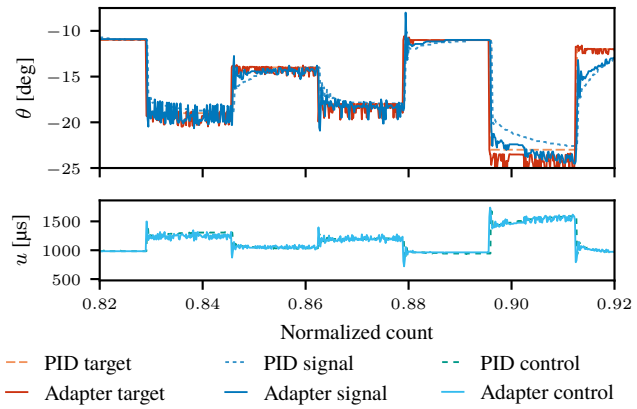
Section VII provided some short interpretation and discussion on the various results presented there. This section will provide insight in the genr=eral trends observed and discuss the limitations of the method that were encountered.

A. Key Insights

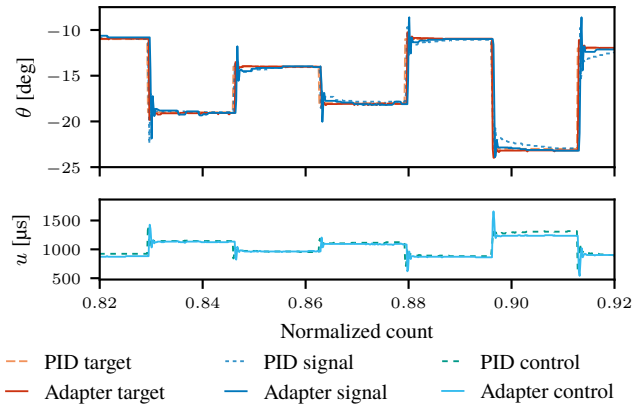
A general trend in the effect of the target adapter can be observed: Whenever a controller-system configuration exhibits under-responsive characteristics, such as the under-tuned controller in Section VII-A and the 0.25–0.25mm & 0.5–0.5mm perforations in Section VII-B, the target adapter induces more aggressive controller behavior. Likewise, when the controller is too aggressive from the start, target adaptions tend to moderate or constrain the behavior.

This follows from the interpretation of the evaluation metrics in these scenarios. For example, under-responsive systems exhibit high rise times, and therefore high settling times. These systems are also characterized by low overshoot, low mean absolute velocity, and low controller *effort* (total variance). When the adapter produces more extreme target positions, rise time and settling time decrease, but the overshoot increases. The controller is thus “making an extra effort”, which is observed in the higher total variance: initially setting a more extreme target position, then gradually bringing it closer to the desired target, etc. The overall faster response results in the increased mean absolute velocity. As can be observed in Figure 24, this behavior resembles that of the under-responsive simulation in the illustrative problem (Section IV-B) very closely.

Similarly, systems with an over-aggressive controller, i.e. the *slow-aggressive* controller experiment and the 2–0.5mm & 1.5–0.5mm perforations, exhibit low rise times, but high



(a)



(b)

Fig. 21: Qualitative comparison of 30s of the signal response for the 2–0.5mm perforation. Also showing a comparison between the signals of the target adapted controller, late vs. early into the experiment.

settling times: the high gains *push* state toward the desired target quickly, and then overcompensate for the high overshoot, resulting in a difficult to maintain a stable response near the target. This explains the high controller *effort*. The adapter moves the target to less extreme positions, compensating for the overshoot "in advance". This prevents the system from entering an unstable feedback loop, by preventing the control inputs from getting overly aggressive in the first place. Thus, even though target adaptions are made, the overall controller effort is decreased. Figure 25 provides a clearer view of the process. Again, a resemblance to the equivalent scenario in Section IV-A can be observed. The contradicting results from the 2–0.5 & 1.5–0.5mm perforations, see Figure 20, will be addressed in Section VIII-B. However, as mentioned in Section VII-B, the adapted controller *does* outperform the reference in the early stages, and it is during this phase that the same trends described earlier can be observed.

These two observed trends in the target adapter's effect suggest that the method is inherently self-correcting. By constantly retraining the adapter on features derived from its own influence on the signal response, an under-responsive adapted controller will become more aggressive, up to the point that the adapter needs to moderate its behavior again. This idea is supported by the observation that the evolution of the metrics for adapted controllers tends to stagnate (most clearly visible in Figures 16, 18 and 19). Supposedly, the adapted controller finds a good compromise between moderate and aggressive behavior, and is able to maintain a consistent performance from that point onward.

Another insight is provided by the difference between the preatrained and online trained adapter in the crawling gait experiment.

discuss diff w/ figures

B. Limitations

As discussed in Section VII-B, the experiment using 1.5–0.5 & 2–0.5mm perforations in combination with a highly responsive PID controller yielded better results for the reference controller than for the adapted controller. The hypothesis is that the training data obtained from the signal response gradually contains more conflicting features, as illustrated by Figure 26. As the controller is highly responsive, the frequency at which the signal response is stored in the buffer is too low to capture sufficient detail.

However, this high responsiveness alone is not solely responsible for the adapter's deteriorating performance, since the adapter initially manages to slightly improve performance. The same reference controller is used in the 0.25–0.25 & 0.5–0.5mm perforation experiments, as well as in the crawling gait experiments (Sections VII-B and VII-C, respectively). It seems to be the specific combination of a highly responsive controller–or conversely, a low sampling rate for the buffer–with a more deteriorated KOS that leads to the issue. Once the system enters an unstable feedback loop, the oscillations occur so frequently that any meaningful relationship between sequential datapoints is lost, an observation which becomes clear by comparing Figures 25 and 26. As a result, training features obtained from near-identical regions in the signal response can vay drastically. The adapter thus starts producing less meaningful adaptations, entering a vicious cycle.

This explains why the combination of 1.5–0.5mm perforations and a *slow*-aggressive controller poses no problem, nor does the same perforation combined with the highly responsive controller during the first 30min (also demonstrated by the crawling gait experiment). Similarly, the 0.25–0.25 and 0.5–0.5mm perforations effectively prevent the controller from entering an unstable state.

C. Validity of the Results

Several important aspects regarding the validity of the results should be addressed.

1) Reproducibility of Identical KOS: Since a key part of understanding the results is the use of a reference controller as a baseline, it is important to confirm that the comparisons are representative. Section VI already stressed that identical target sequences were used for each comparison. The focus thus shifts to the possible differences in the characteristics of supposedly identical KOSs.

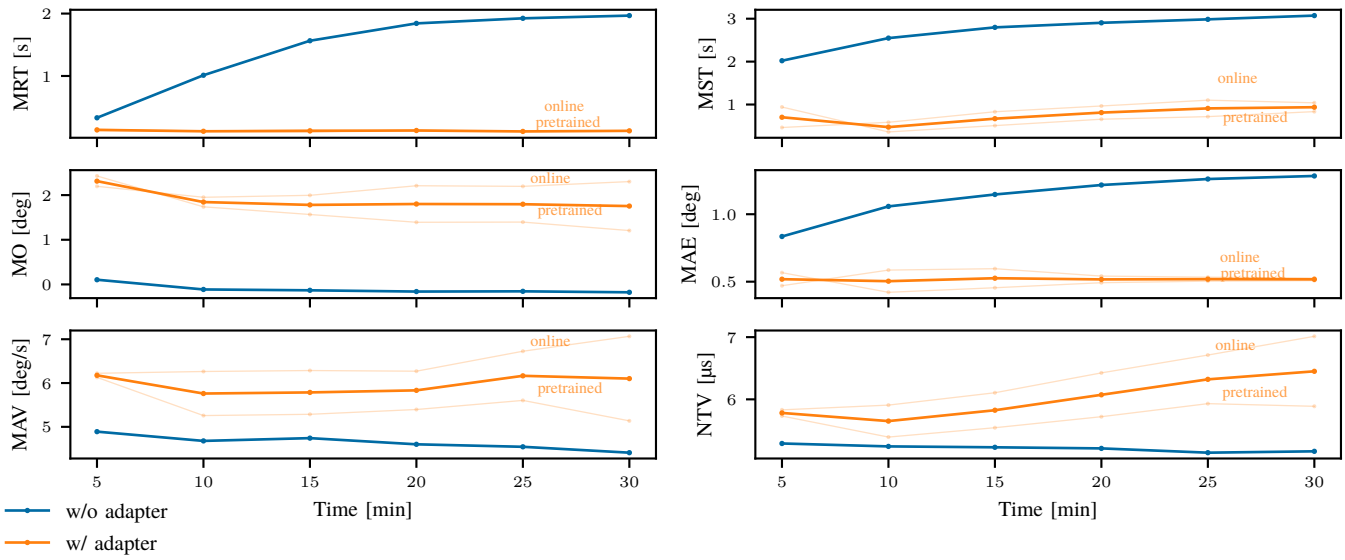


Fig. 22: Metrics evolution over the 20min duration of the crawling-gait experiment, with the average of both adapted controllers overlayed. Each point represents the calculated value of the metric on identical 5min sequences of targets, in this case alternating between values near to -18 and -8 degrees, respectively. Plots for all metrics and numerical values can be found in Figure B.5 and Table B.V, respectively.

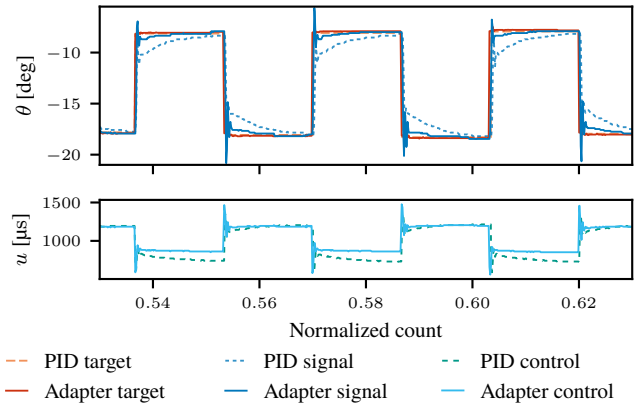


Fig. 23

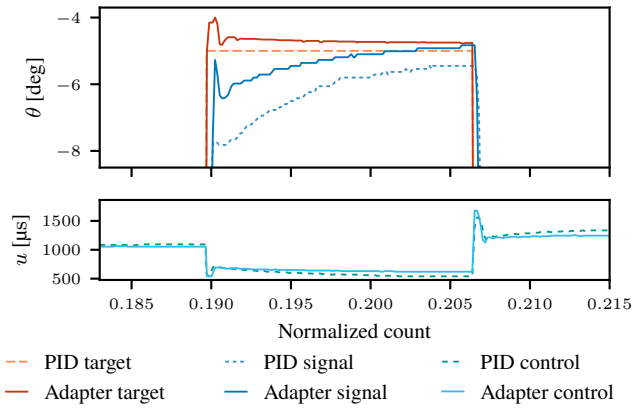


Fig. 24: A close-up from the 0.5–0.5mm perforation with target adapter. The signal response generally resembles that of Figure 10 from the toy problem.

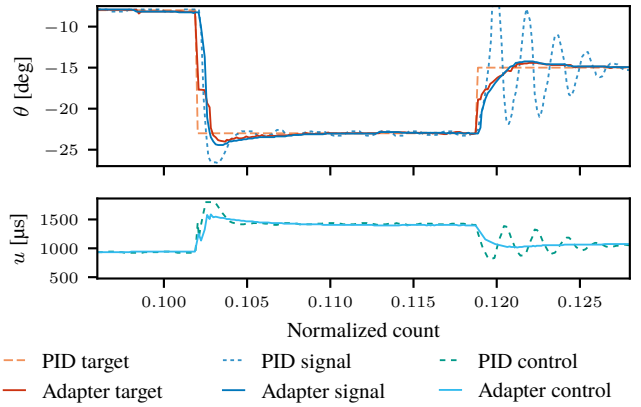


Fig. 25: A close-up from the *slow-aggressive* PID controller with target adapter. The manner in which the target is adapted resembles that of Figure 8 from the toy problem.

Fig. 26: A close-up of the 2–0.5mm experiment. The oscillations in the signal response are recorded at an insufficient frequency to contain any meaningful relationship between spaced-out state pairs and the target. As a result, the adapter's output becomes more chaotic as well, further polluting the training data.

The KOS templates were all designed with identical parameters, exempting the perforations, and were cut out using a laser-cutter. They were, however, assembled by hand. Several steps were taken to mitigate or minimize variability introduced during the assembly.

First, the required KOSs for a certain experiment setting–

usually four, since two are required for the reference run, and 2 for the adapter test–were all made in a single *batch*, following a preset routine. Additionally, all KOSs were then "conditioned" by exactly one cycle of contracting and expanding it. This procedure reduces potential differences in the cutting and assembly process.

Second, the nature of the robot's design requires two KOSs per experiment run: as described in Section V-A, the state of the robot is represented by the angle of the actuation arm. This arm is restrained on both sides by a KOS, each a mirror image of the other. In practice, the combined contribution of both springs averages out, and the influence of any single KOS is minimized.

Finally, during each run of an experiment setting, a "reference minute" was recorded with intervals of 30min. During these reference minutes, and regardless of the specific experiment setting (i.e., with or without the adapter), the robot ran the relevant reference controller on a 1min target sequence. These minutes were evaluated using the same metrics as the experiments to compare the similarity of the KOSs involved. Comparisons of these metrics are presented in Appendix C, but it is sufficient here to note that variations in metric values for reference minutes are insignificant compared to the difference introduced by the implementation of the target adapter.

These measures allow for valid interpretation of the results.

2) Uninterpreted Results: For certain experiments, the nature of the adapter's influence causing the improvement remains unclear. Qualitative results for both adapter configurations of the crawling gait experiment, and the early stage of the 1.5–0.5 & 2–0.5mm perforation experiment (Figures 21 and 23) do not indicate significant adaptations to the target. This might lead to disregarding the results as coincidences. However, the improvement is consistent over all these experiment settings, with notable similarity in the reference signal responses for all settings, as well as adapter signal responses (i.e., signals from seemingly slightly under-responsive systems, and highly responsive systems, respectively).

IX. Conclusion

The goal of the target adaptation approach is to provide a simplified and intuitive alternative to more opaque control techniques commonly seen in soft-robotics applications. The idea is that a simple or traditional method can be used to handle the base control of the robot, and that a separate module improves the performance of that controller. It achieves this by providing the controller with adapted targets, but without altering the controller's parameters.

In simulation, it was shown that such an approach offers near instantaneous improvement for the control of a simple mass-spring damper system.

The tests on a demonstrator robot, with a design relying on Kresling origami springs, show that the method is robust to varying controller tuning and varying system dynamics. The target adapter module tends to make passive controllers more aggressive, and vice versa, resulting in a consistent performance. Usage of the method on a constrained operational domain shows that both pretrained and online trained adapters manage to improve the performance, with a preference for the latter.

A limitation is presented in the form of the sampling frequency at which the signal response is stored for training the module: The frequency has to be high enough to capture enough detail. If the sampling frequency is too low *and* the recorded data contains some unstable oscillations, the adapter is unable to find meaningful relationships between the state pairs and targets it uses for updating itself. Updates on such meaningless features lead to a negative feedback loop for the adapter's performance, which it is unable to escape.

Possible solutions to the problem are methods that allow the adapter to either recall previous settings or maintain a more stable performance. An implementation of some kind of memory, or more envolved updating techniques like *learning without forgetting* (Z. Li & Hoiem, 2018), could potentially allow the adapter to escape such vicious cycles.

filter through refrences, just the essentials

References

Annaswamy, A. M., & Fradkov, A. L. (2021). A Historical Perspective of Adaptive Control and Learning. *Annual Reviews in Control*, *52*, 18–41. https://doi.org/10.1016/j.arcontrol.2021.10.014

Arduino. (2024). Servo Reference. Retrieved September 24, 2024, from https://www.arduino.cc/reference/en/libraries/servo/writemicroseconds/

Bhagat, S., Banerjee, H., Tse, Z. T. H., & Ren, H. (2019, January). Deep reinforcement learning for soft, flexible robots: Brief review with impending challenges. https://doi.org/10.3390/robotics8010004
Meta-studie.

Calisti, M., Picardi, G., & Laschi, C. (2017, May). Fundamentals of soft robot locomotion. https://doi.org/10.1098/rsif.2017.0101

Gillespie, M. T., Best, C. M., Townsend, E. C., Wingate, D., & Killpack, M. D. (2018). Learning nonlinear dynamic models of soft robots for model predictive control with neural networks. *2018 IEEE International Conference on Soft Robotics (RoboSoft)*, 39–45. https://doi.org/10.1109/ROBOSOFT.2018.8404894

Hawkes, E. W., Blumenschein, L. H., Greer, J. D., & Okamura, A. M. (2017). A soft robot that navigates its environment through growth. *Science Robotics*, *2*(8), 1–8. https://doi.org/10.1126/scirobotics.aan3028

Kim, D., Kim, S.-H., Kim, T., Kang, B. B., Lee, M., Park, W., Ku, S., Kim, D., Kwon, J., Lee, H., Bae, J., Park, Y.-L., Cho, K.-J., & Jo, S. (2021). Review of machine learning methods in soft robotics (G. Gu, Ed.). *PLOS ONE*, *16*(2), e0246102. https://doi.org/10.1371/journal.pone.0246102

Kresling, B. (2008). Natural twist buckling in shells: from the hawkmoth's bellows to the deployable Kresling-pattern and cylindrical Miura-ori. *Proceedings of the 6th International Conference on Computation of Shell and Spatial Structures IASS-IACM 2008: "Spanning Nano to Mega"*, (May), 1–4.

Li, R. J., & Han, Z. Z. (2005). Survey of iterative learning control. *Kongzhi yu Juece/Control and Decision*, *20*(9), 961–966. https://doi.org/10.1109/mcs.2006.1636313

Li, Z., & Hoiem, D. (2018). Learning without Forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *40*(12), 2935–2947. https://doi.org/10.1109/TPAMI.2017.2773081

Masana, R., Dalaq, A. S., Khazaaleh, S., & Daqaq, M. F. (2024). The Kresling Origami Spring : A Review and Assessment. *Smart Materials and Structures*, *33*. https://doi.org/10.1088/1361-665X/AD2F6F

Masana, R., & Daqaq, M. F. (2019). Equilibria and bifurcations of a foldable paper-based spring inspired by Kresling-pattern origami. *Physical Review E*, *100*(6), 063001. https://doi.org/10.1103/PhysRevE.100.063001

Pagano, A., Yan, T., Chien, B., Wissa, A., & Tawfick, S. (2017). A crawling robot driven by multi-stable origami. *Smart Materials and Structures*, *26*(9). https://doi.org/10.1088/1361-665X/aa721e

Rob Tillaart. (2024). Arduino library for AS5600 magnetic rotation meter. Retrieved September 24, 2024, from https://github.com/RobTillaart/AS5600

Tang, J., & Wei, F. (2022). Miniaturized Origami Robots: Actuation Approaches and Potential Applications. *Macromolecular Materials and Engineering*, *307*(2), 2100671. https://doi.org/10.1002/mame.202100671

Thuruthel, T. G., Falotico, E., Renda, F., & Laschi, C. (2019). Model-Based Reinforcement Learning for Closed-Loop Dynamic Control of Soft Robotic Manipulators. *IEEE Transactions on Robotics*, *35*(1), 124–134. https://doi.org/10.1109/TRO.2018.2878318

Wang, Z., Chen, C., & Dong, D. (2020). Lifelong Incremental Reinforcement Learning with Online Bayesian Inference. *IEEE Transactions on Neural Networks and Learning Systems*, *33*(8), 4003–4016. https://doi.org/10.1109/TNNLS.2021.3055499

Ze, Q., Wu, S., Nishikawa, J., Dai, J., Sun, Y., Leanza, S., Zemelka, C., Novelino, L. S., Paulino, G. H., & Zhao, R. R. (2022). Soft robotic origami crawler. *Science Advances*, *8*(13), 7834. https://doi.org/10.1126/sciadv.abm7834

Appendix A
Software implementation

The supervised learning implementation for target adaptations is divided into two key components: the nominal control of the robot, programmed in C++ and executed by the Arduino board, and the target adaptation, implemented in Python and managed by an external computer. The adapter is implemented and trained using Keras with the PyTorch backend.

A. Nominal Control on the Arduino

Upon startup, the Arduino initializes a connection with the external computer and directs the servo to move to its minimum position. The Arduino allows some time for the servo to reach this home position and then reads a first measurement from the angle sensor, setting this as the reference $\theta = 0$. During the control loop, the servo reads θ (relative to the reference) and $\dot{\theta}$ received from the sensor, and reads the target θ_d received from the external computer. Based on these measurements, the PID controller computes the value to be written to the servo, but before writing it to the servo, a ceiling is imposed on it to protect the servo from exceeding its mechanical limit (Section V-A). The Arduino allows some time for the servo to act and then reads the measurements from the angle sensor again. Finally, it sends the tuple containing the state, action, resulting state, and target to the external computer.

B. Target Adaptation on the External Computer

The target adaptation and supervised learning are implemented exclusively on the external computer through the established connection with the Arduino. The program is divided into several modules or *threads* that run simultaneously. This way, values that need to be sent or read can be updated in the background upon request, without delaying the operations of other modules (see Appendix A for a more detailed breakdown of the interactions between various threads). At regular predefined intervals, a `change_value` thread changes the true target θ_d to some value. Every iteration, it constructs an input feature $[\theta, \dot{\theta}, \theta_d, 0]$ for the adapter. The resulting Δ is added to the target, which is read by the `send` thread and forwarded to the Arduino. The buffer is maintained by storing the tuples received from the Arduino. Upon train-time, the adapter weights are updated in the background, without interrupting the adapter's performance.

Figure A.1 provides an overview of the communication between computer and Arduino, and their inner interactions between various modules. For example, the sole task of the `send` thread is to read the value of a global variable representing the target state and send this to the Arduino. The `listen` thread reads the tuple sent in return by the Arduino and assigns its contents to the respective shared variables for the state, control action, resulting state and target. It then adds this tuple to the buffer used for updating the adapter model. The `update` thread is responsible for (re)training the adapter. It creates a copy of the adapter, which can then be trained in the background while the original adapter remains functional. Once training is completed, the copied adapter's weights are assigned to the original adapter. The `change_value` thread is responsible for setting the "true" target in the control loop, as well as the actual adaptations to it. All threads aim to operate at 50Hz.

Appendix B
Extended Test Results

Appendix C
Comparing KOSs

ref minute plots and tables

TABLE B.I: Numerical values of the metrics for the *slow-aggressive* controller.

(a) Baseline controller

Metric	5	10	15	20	25	30	35	40	45	50	55	60
MRT [s]	0.15	0.15	0.15	0.16	0.17	0.15	0.15	0.14	0.15	0.29	0.36	0.44
MST [s]	3.02	2.88	2.73	2.63	2.71	2.62	2.92	2.76	2.33	2.47	2.68	2.72
MO [deg]	3.91	3.62	3.45	3.35	3.33	3.31	3.39	3.45	3.07	2.85	2.73	2.52
MAE [deg]	1.28	1.23	1.23	1.13	1.13	1.08	0.94	1.08	0.93	1.00	1.04	1.00
MAV [deg/s]	16.02	15.61	15.19	14.05	13.85	12.74	12.16	12.84	11.33	10.96	11.17	10.60
NTV [µs]	10.83	10.69	10.76	9.82	9.67	9.37	8.24	9.53	8.16	8.22	8.08	7.64

Metric	65	70	75	80	85	90	95	100	105	110	115	120
MRT [s]	0.14	0.14	0.16	0.35	0.74	0.80	0.16	0.31	0.52	0.54	0.50	0.56
MST [s]	3.05	2.51	2.30	2.28	2.61	2.60	2.50	2.55	2.50	2.53	2.49	2.17
MO [deg]	3.41	3.08	2.97	2.60	2.22	2.07	2.50	2.26	2.08	2.00	2.17	2.01
MAE [deg]	1.04	0.84	0.78	0.82	1.01	1.05	0.76	0.78	0.84	0.84	0.82	0.82
MAV [deg/s]	12.09	9.09	9.14	8.47	7.90	7.35	8.78	7.96	7.03	7.00	7.20	6.50
NTV [µs]	8.99	7.40	6.84	6.26	5.95	5.84	6.69	6.08	5.71	5.69	5.62	5.50

(b) Target adapter

Metric	5	10	15	20	25	30	35	40	45	50	55	60
MRT [s]	0.42	0.61	0.19	0.17	0.18	0.19	0.18	0.17	0.19	0.20	0.21	0.17
MST [s]	2.79	3.15	2.48	2.42	2.39	2.50	2.18	2.23	2.51	2.36	2.27	2.36
MO [deg]	2.26	2.08	3.08	3.00	2.86	2.93	2.91	2.93	2.88	2.85	2.90	2.85
MAE [deg]	0.82	1.00	0.74	0.62	0.63	0.62	0.64	0.62	0.63	0.61	0.61	0.61
MAV [deg/s]	7.33	8.12	7.74	6.95	6.51	6.47	6.77	6.59	6.68	6.46	6.46	5.92
NTV [µs]	5.33	6.33	5.70	4.84	4.57	4.81	4.77	4.77	4.77	4.71	4.66	4.58

Metric	65	70	75	80	85	90	95	100	105	110	115	120
MRT [s]	0.17	0.16	0.19	0.17	0.20	0.15	0.16	0.17	0.18	0.20	0.21	0.22
MST [s]	2.33	2.23	2.41	2.33	2.17	2.14	2.36	2.30	2.38	2.32	2.43	2.38
MO [deg]	2.98	2.92	2.71	2.82	2.82	2.77	2.99	2.85	2.84	2.85	2.85	2.85
MAE [deg]	0.63	0.60	0.61	0.61	0.60	0.57	0.64	0.60	0.60	0.60	0.62	0.60
MAV [deg/s]	6.37	5.97	6.59	5.90	6.33	5.92	6.53	6.22	6.20	6.23	6.45	6.08
NTV [µs]	4.80	4.70	4.77	4.68	4.57	4.47	4.86	4.62	4.74	4.79	4.85	4.81

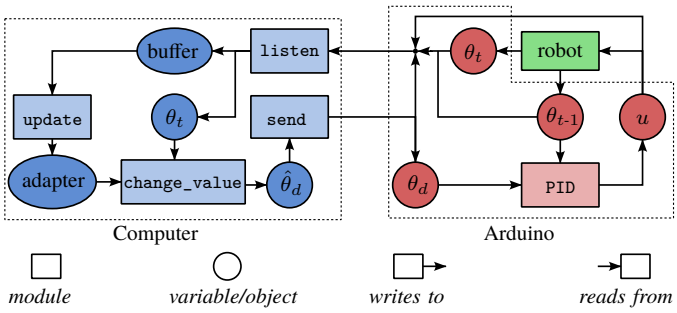


Fig. A.1: The interaction between the modules (rectangles) and shared variables (ovals) of the computer (blue) and Arduino (red).

Appendix D
KOS Design and Template

The KOSs used during the experiments used the following design parameters, as related to the definition from Section II-C:

$$\begin{aligned} a_0 &\quad 25\text{mm} \\ a_0/b_0 &\quad 1.5 \\ \alpha_0 &\quad 28 \text{ degrees} \end{aligned}$$

A template for constructing this KOS can be found at the end of the document.

Instructions:

1) Tear out the template along its outer contours.
2) Fold the Kresling pattern: The parallelogram's diagonals are valley folds, the sides are mountain folds.
3) Form a cylinder and glue the flaps at the ends to the inside of their respective counterpart.
4) Insert the extensions at the top and bottom of the cylinder through the slits in the two base hexagons.
5) Fold the extensions outward and glue them to the bases.
6) For extra strength, an extra (cardboard) hexagonal base can be glued to the top and bottom of the KOS.

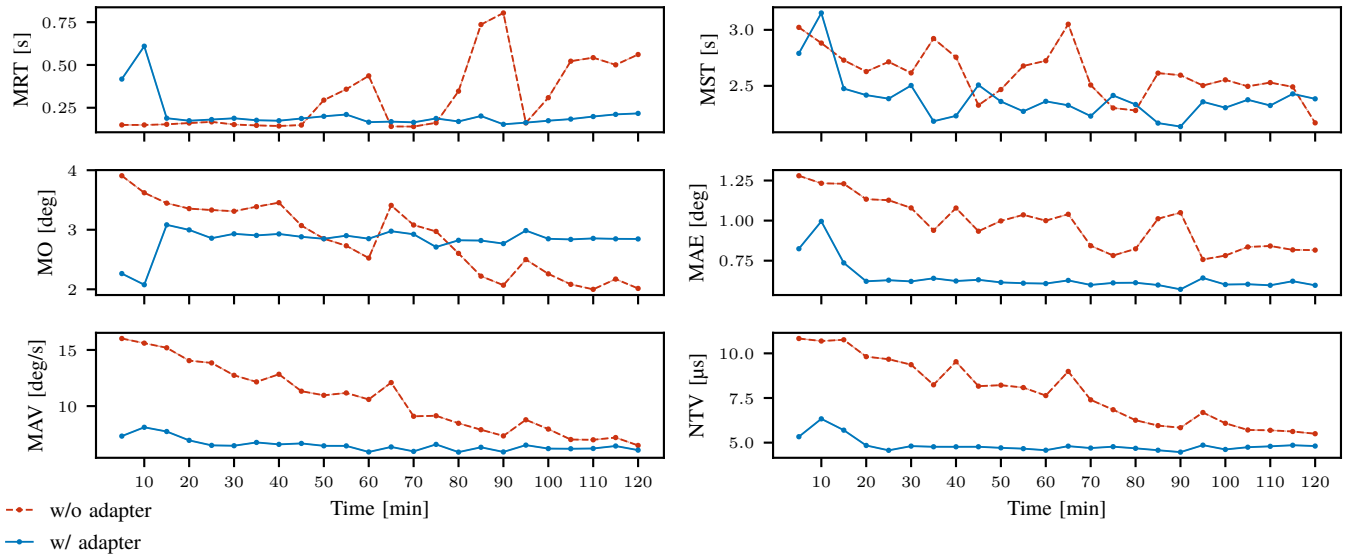


Fig. B.1: Metrics evolution for the *slow-aggressive* controller.

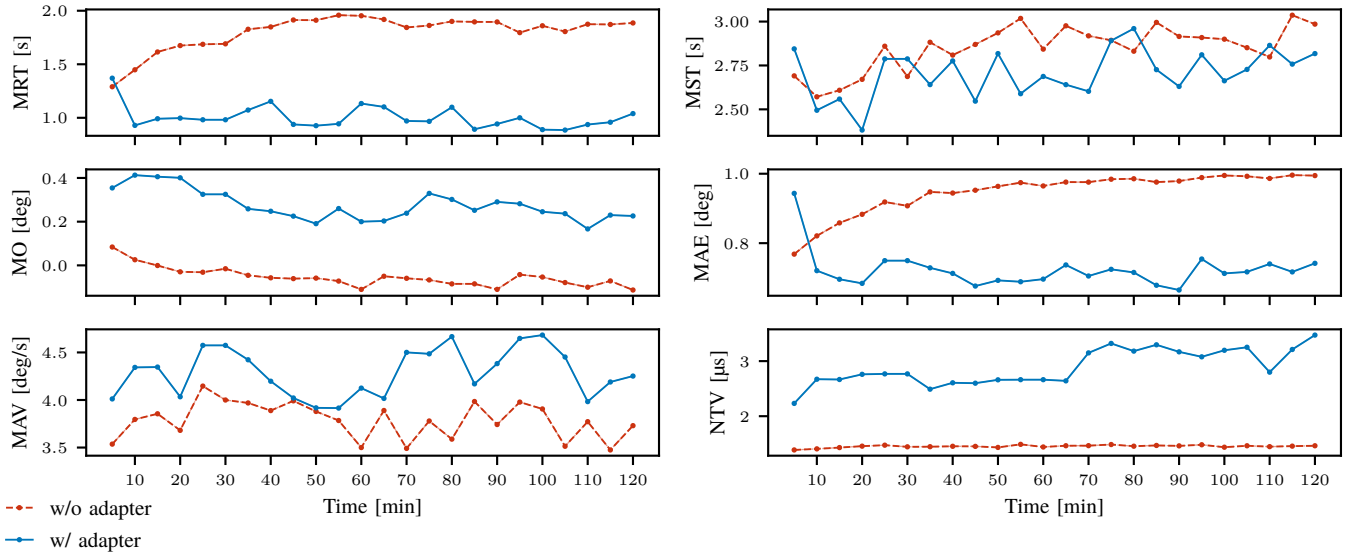


Fig. B.2: Metrics evolution for the *under-tuned* controller.

TABLE B.II: Numerical values of the metrics for the *under-tuned* controller.

(a) Baseline controller

Metric	5	10	15	20	25	30	35	40	45	50	55	60
MRT [s]	1.29	1.45	1.61	1.67	1.69	1.69	1.83	1.85	1.91	1.91	1.96	1.95
MST [s]	2.69	2.57	2.61	2.67	2.86	2.69	2.88	2.81	2.87	2.94	3.02	2.84
MO [deg]	0.08	0.03	-0.00	-0.03	-0.03	-0.02	-0.05	-0.06	-0.06	-0.06	-0.07	-0.11
MAE [deg]	0.77	0.82	0.86	0.88	0.92	0.91	0.95	0.94	0.95	0.96	0.97	0.96
MAV [deg/s]	3.54	3.80	3.86	3.68	4.15	4.00	3.97	3.89	3.99	3.88	3.78	3.50
NTV [µs]	1.39	1.41	1.43	1.46	1.47	1.44	1.45	1.45	1.45	1.43	1.49	1.44

Metric	65	70	75	80	85	90	95	100	105	110	115	120
MRT [s]	1.92	1.84	1.86	1.90	1.90	1.90	1.80	1.86	1.81	1.87	1.87	1.89
MST [s]	2.98	2.92	2.89	2.83	3.00	2.92	2.91	2.90	2.85	2.80	3.04	2.99
MO [deg]	-0.05	-0.06	-0.07	-0.08	-0.08	-0.11	-0.04	-0.05	-0.08	-0.10	-0.07	-0.11
MAE [deg]	0.98	0.98	0.98	0.99	0.98	0.98	0.99	0.99	0.99	0.99	1.00	0.99
MAV [deg/s]	3.89	3.49	3.78	3.59	3.98	3.74	3.98	3.91	3.51	3.77	3.47	3.73
NTV [µs]	1.46	1.46	1.49	1.46	1.47	1.46	1.48	1.44	1.46	1.45	1.46	1.46

(b) Target adapter

Metric	5	10	15	20	25	30	35	40	45	50	55	60
MRT [s]	1.37	0.93	0.99	1.00	0.98	0.98	1.07	1.15	0.94	0.93	0.94	1.13
MST [s]	2.84	2.49	2.56	2.38	2.79	2.79	2.64	2.78	2.55	2.82	2.59	2.69
MO [deg]	0.35	0.41	0.41	0.40	0.33	0.33	0.26	0.25	0.23	0.19	0.26	0.20
MAE [deg]	0.94	0.72	0.70	0.68	0.75	0.75	0.73	0.71	0.68	0.69	0.69	0.70
MAV [deg/s]	4.01	4.34	4.35	4.03	4.57	4.57	4.42	4.20	4.02	3.92	3.92	4.12
NTV [µs]	2.23	2.67	2.67	2.76	2.77	2.77	2.49	2.61	2.60	2.66	2.66	2.66

Metric	65	70	75	80	85	90	95	100	105	110	115	120
MRT [s]	1.10	0.97	0.97	1.10	0.89	0.94	1.00	0.89	0.89	0.94	0.96	1.04
MST [s]	2.64	2.60	2.89	2.96	2.73	2.63	2.81	2.66	2.73	2.86	2.76	2.82
MO [deg]	0.20	0.24	0.33	0.30	0.25	0.29	0.28	0.25	0.24	0.17	0.23	0.23
MAE [deg]	0.74	0.71	0.72	0.72	0.68	0.67	0.75	0.71	0.72	0.74	0.72	0.74
MAV [deg/s]	4.02	4.50	4.49	4.67	4.17	4.38	4.65	4.68	4.45	3.98	4.19	4.25
NTV [µs]	2.64	3.15	3.32	3.18	3.30	3.17	3.08	3.20	3.25	2.80	3.21	3.47

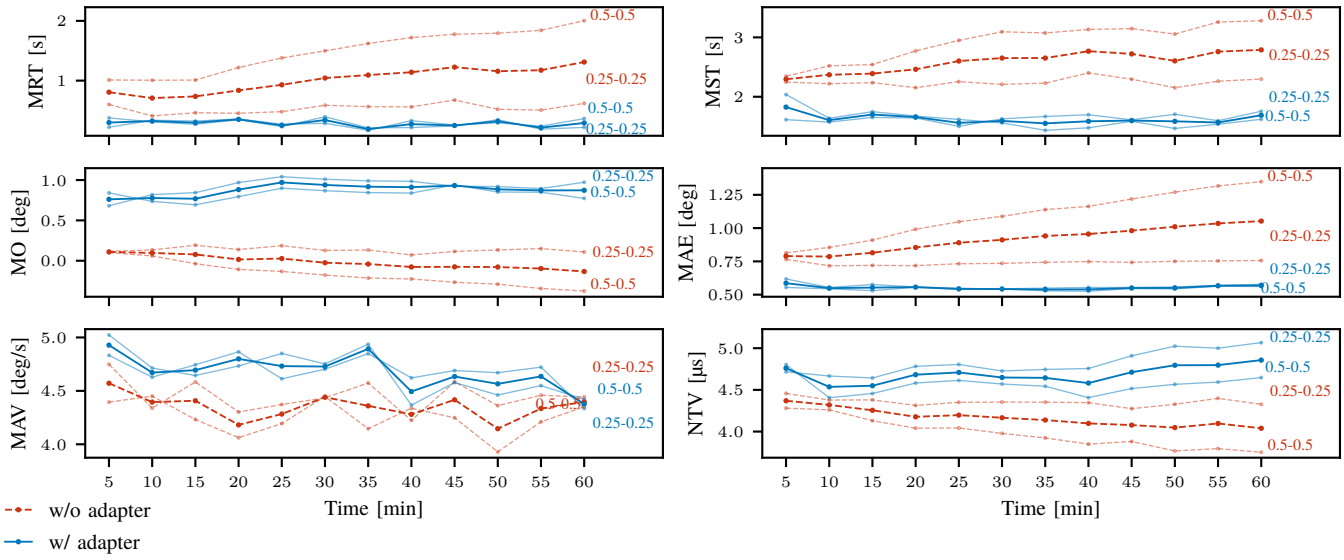


Fig. B.3: Metrics evolution for the 25–25 and 50–50mm perforations.

TABLE B.III: Numerical values of the metrics for the 25–25 and 50–50mm perforations.

(a) 25–25mm Baseline controller

Metric	5	10	15	20	25	30	35	40	45	50	55	60
MRT [s]	0.60	0.41	0.46	0.45	0.48	0.59	0.56	0.56	0.68	0.52	0.51	0.62
MST [s]	2.25	2.22	2.24	2.15	2.25	2.21	2.23	2.40	2.30	2.15	2.26	2.30
MO [deg]	0.12	0.13	0.19	0.14	0.19	0.13	0.13	0.07	0.12	0.13	0.15	0.11
MAE [deg]	0.77	0.72	0.72	0.72	0.73	0.74	0.74	0.75	0.74	0.75	0.75	0.76
MAV [deg/s]	4.75	4.34	4.58	4.30	4.37	4.43	4.57	4.22	4.58	4.36	4.46	4.44
NTV [µs]	4.46	4.38	4.38	4.31	4.35	4.35	4.35	4.35	4.27	4.33	4.40	4.33

(b) 25–25mm Target adapter

Metric	5	10	15	20	25	30	35	40	45	50	55	60
MRT [s]	0.38	0.31	0.27	0.35	0.23	0.39	0.21	0.22	0.24	0.35	0.19	0.22
MST [s]	1.61	1.57	1.65	1.64	1.50	1.63	1.67	1.70	1.61	1.71	1.60	1.76
MO [deg]	0.68	0.82	0.84	0.97	1.04	1.01	0.99	0.98	0.92	0.92	0.89	0.97
MAE [deg]	0.55	0.54	0.53	0.56	0.54	0.54	0.55	0.55	0.55	0.56	0.57	0.58
MAV [deg/s]	4.83	4.63	4.74	4.87	4.61	4.70	4.85	4.62	4.69	4.67	4.72	4.33
NTV [µs]	4.72	4.67	4.64	4.78	4.80	4.73	4.74	4.76	4.91	5.02	5.00	5.07

(c) 50–50mm Baseline controller

Metric	5	10	15	20	25	30	35	40	45	50	55	60
MRT [s]	1.01	1.01	1.01	1.22	1.38	1.50	1.62	1.72	1.78	1.79	1.84	2.00
MST [s]	2.34	2.52	2.54	2.77	2.95	3.09	3.07	3.13	3.15	3.06	3.26	3.28
MO [deg]	0.10	0.06	-0.04	-0.11	-0.13	-0.18	-0.21	-0.23	-0.27	-0.29	-0.34	-0.38
MAE [deg]	0.81	0.85	0.91	0.99	1.05	1.09	1.14	1.16	1.22	1.27	1.32	1.35
MAV [deg/s]	4.39	4.45	4.23	4.06	4.19	4.45	4.15	4.34	4.25	3.93	4.21	4.35
NTV [µs]	4.28	4.26	4.13	4.04	4.04	3.98	3.92	3.85	3.88	3.77	3.80	3.75

(d) 50–50mm Target adapter

Metric	5	10	15	20	25	30	35	40	45	50	55	60
MRT [s]	0.22	0.34	0.32	0.36	0.27	0.29	0.17	0.33	0.26	0.29	0.24	0.36
MST [s]	2.04	1.64	1.75	1.67	1.62	1.56	1.44	1.48	1.59	1.47	1.54	1.62
MO [deg]	0.84	0.74	0.69	0.79	0.90	0.87	0.85	0.84	0.94	0.85	0.85	0.77
MAE [deg]	0.62	0.55	0.58	0.56	0.55	0.54	0.53	0.53	0.55	0.54	0.56	0.56
MAV [deg/s]	5.02	4.71	4.64	4.73	4.85	4.75	4.94	4.37	4.58	4.46	4.55	4.42
NTV [µs]	4.80	4.41	4.46	4.58	4.61	4.57	4.54	4.41	4.52	4.57	4.59	4.65

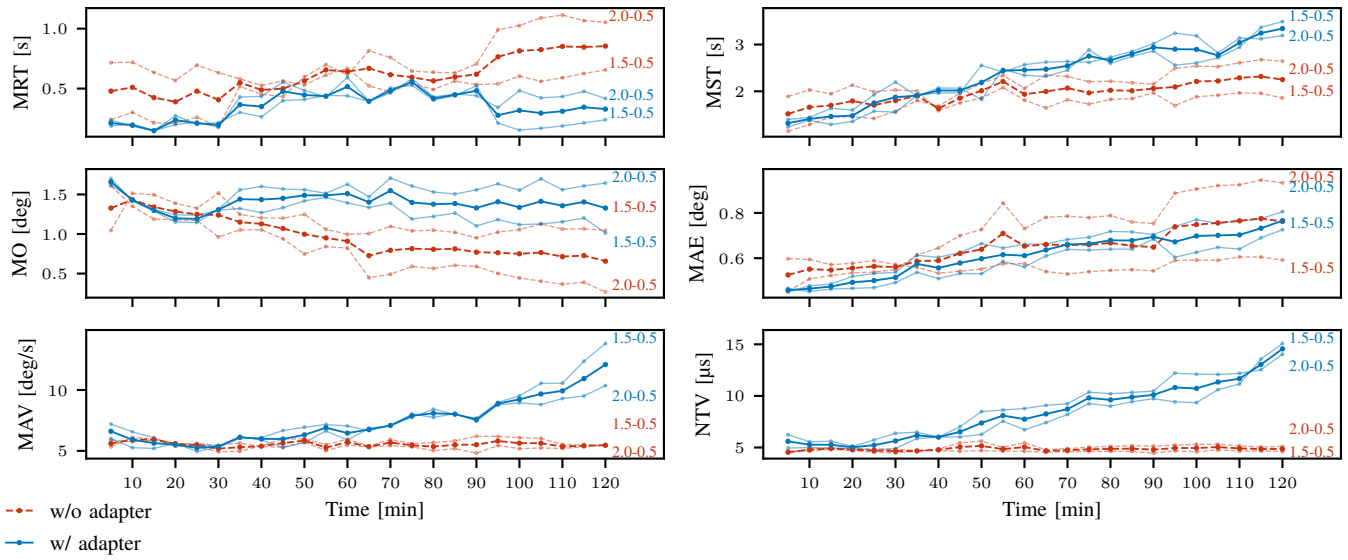


Fig. B.4: Metrics evolution for the 150–50 and 200–50mm perforations.

TABLE B.IV: Numerical values of the metrics for the 150–50 and 200–50mm perforations.

(a) 150–50mm Baseline controller

Metric	5	10	15	20	25	30	35	40	45	50	55	60
MRT [s]	0.72	0.72	0.63	0.57	0.70	0.63	0.58	0.53	0.56	0.53	0.61	0.67
MST [s]	1.89	2.03	1.95	2.13	1.99	2.03	2.01	1.59	1.75	1.86	2.08	1.81
MO [deg]	1.05	1.52	1.50	1.39	1.33	1.52	1.25	1.20	1.20	1.25	1.06	1.00
MAE [deg]	0.60	0.59	0.57	0.58	0.59	0.57	0.56	0.53	0.54	0.55	0.58	0.58
MAV [deg/s]	5.33	6.11	5.98	5.57	5.60	5.45	5.63	5.36	5.29	5.72	5.02	5.48
NTV [µs]	4.46	4.90	4.96	4.87	4.87	4.84	4.70	4.70	4.64	4.70	4.65	4.64

Metric	65	70	75	80	85	90	95	100	105	110	115	120
MRT [s]	0.52	0.47	0.54	0.49	0.56	0.53	0.54	0.60	0.56	0.59	0.63	0.66
MST [s]	1.65	1.82	1.73	1.83	1.84	1.97	1.70	1.88	1.92	1.97	1.96	1.86
MO [deg]	1.01	1.10	1.04	1.05	1.02	0.95	1.02	1.06	1.13	1.06	1.07	1.05
MAE [deg]	0.54	0.53	0.54	0.55	0.55	0.54	0.59	0.59	0.59	0.61	0.61	0.59
MAV [deg/s]	5.31	5.43	5.32	5.02	5.18	4.81	5.44	5.18	5.23	5.19	5.34	5.52
NTV [µs]	4.54	4.59	4.62	4.63	4.61	4.46	4.67	4.59	4.76	4.65	4.66	4.62

(b) 150–50mm Target adapter

Metric	5	10	15	20	25	30	35	40	45	50	55	60
MRT [s]	0.19	0.20	0.15	0.27	0.20	0.22	0.30	0.26	0.40	0.41	0.44	0.44
MST [s]	1.40	1.44	1.63	1.60	1.93	2.20	1.90	2.07	2.07	1.83	2.47	2.34
MO [deg]	1.62	1.44	1.31	1.24	1.24	1.30	1.32	1.27	1.33	1.42	1.46	1.39
MAE [deg]	0.47	0.45	0.46	0.47	0.47	0.49	0.54	0.51	0.53	0.53	0.59	0.56
MAV [deg/s]	7.20	6.56	6.08	5.38	5.55	5.39	6.18	5.90	5.28	5.68	6.64	5.88
NTV [µs]	6.24	5.55	5.61	5.10	5.73	6.38	6.49	6.02	6.02	6.27	7.55	6.72

Metric	65	70	75	80	85	90	95	100	105	110	115	120
MRT [s]	0.39	0.50	0.53	0.40	0.44	0.52	0.21	0.15	0.17	0.19	0.21	0.24
MST [s]	2.32	2.45	2.89	2.60	2.75	2.86	2.56	2.61	2.72	2.94	3.37	3.49
MO [deg]	1.33	1.39	1.19	1.22	1.27	1.10	1.18	1.12	1.13	1.15	1.20	1.01
MAE [deg]	0.61	0.64	0.64	0.64	0.64	0.68	0.60	0.63	0.65	0.64	0.69	0.73
MAV [deg/s]	6.85	7.07	7.76	8.42	7.98	7.67	8.98	9.52	10.55	10.56	12.36	13.82
NTV [µs]	7.42	8.20	9.25	9.02	9.43	9.73	9.42	9.35	10.61	11.16	13.57	15.08

(c) 200–50mm Baseline controller

Metric	5	10	15	20	25	30	35	40	45	50	55	60
MRT [s]	0.24	0.30	0.21	0.21	0.26	0.18	0.52	0.45	0.43	0.60	0.70	0.61
MST [s]	1.15	1.29	1.45	1.45	1.42	1.57	1.80	1.68	1.96	2.17	2.34	2.06
MO [deg]	1.61	1.35	1.19	1.18	1.17	0.96	1.05	1.05	0.94	0.75	0.84	0.82
MAE [deg]	0.45	0.51	0.52	0.54	0.54	0.55	0.61	0.65	0.70	0.73	0.84	0.73
MAV [deg/s]	5.87	5.74	5.87	5.60	5.34	4.93	4.97	5.41	5.98	5.98	5.51	5.96
NTV [µs]	4.64	4.66	4.84	4.71	4.54	4.46	4.64	4.87	5.45	5.62	5.02	5.45

Metric	65	70	75	80	85	90	95	100	105	110	115	120
MRT [s]	0.82	0.76	0.65	0.64	0.63	0.71	0.99	1.03	1.09	1.11	1.07	1.05
MST [s]	2.35	2.32	2.21	2.22	2.18	2.15	2.49	2.54	2.53	2.61	2.67	2.64
MO [deg]	0.45	0.49	0.59	0.56	0.60	0.59	0.50	0.44	0.40	0.37	0.39	0.27
MAE [deg]	0.78	0.79	0.78	0.79	0.76	0.75	0.89	0.91	0.92	0.92	0.95	0.93
MAV [deg/s]	5.39	5.94	5.59	5.68	5.81	6.20	6.18	6.08	6.02	5.52	5.49	5.39
NTV [µs]	4.79	4.89	5.02	5.10	5.17	5.13	5.23	5.31	5.32	5.17	5.04	5.10

(d) 200–50mm Target adapter

Metric	5	10	15	20	25	30	35	40	45	50	55	60
MRT [s]	0.24	0.19	0.15	0.20	0.22	0.18	0.43	0.44	0.55	0.48	0.43	0.59
MST [s]	1.24	1.38	1.29	1.36	1.57	1.55	1.92	1.96	1.97	2.55	2.41	2.57
MO [deg]	1.70	1.42	1.29	1.15	1.15	1.31	1.56	1.60	1.57	1.56	1.51	1.63
MAE [deg]	0.45	0.48	0.49	0.52	0.53	0.54	0.61	0.60	0.63	0.66	0.65	0.66
MAV [deg/s]	6.01	5.26	5.20	5.62	4.95	5.31	6.05	6.08	6.67	6.94	7.16	7.04
NTV [µs]	4.95	4.99	4.93	4.94	4.72	4.90	5.87	5.99	7.02	8.48	8.63	8.78

Metric	65	70	75	80	85	90	95	100	105	110	115	120
MRT [s]	0.39	0.46	0.58	0.43	0.45	0.44	0.34	0.48	0.42	0.43	0.48	0.42
MST [s]	2.62	2.64	2.62	2.73	2.85	3.02	3.24	3.19	2.82	3.14	3.12	3.19
MO [deg]	1.47	1.71	1.61	1.53	1.51	1.56	1.63	1.55	1.70	1.56	1.61	1.64
MAE [deg]	0.66	0.68	0.69	0.72	0.72	0.70	0.74	0.77	0.75	0.77	0.77	0.81
MAV [deg/s]	6.67	7.09	8.02	7.75	8.05	7.46	8.77	8.95	8.80	9.30	9.50	10.36
NTV [µs]	9.09	9.24	10.37	10.21	10.34	10.47	12.21	12.10	12.08	12.18	12.53	14.04

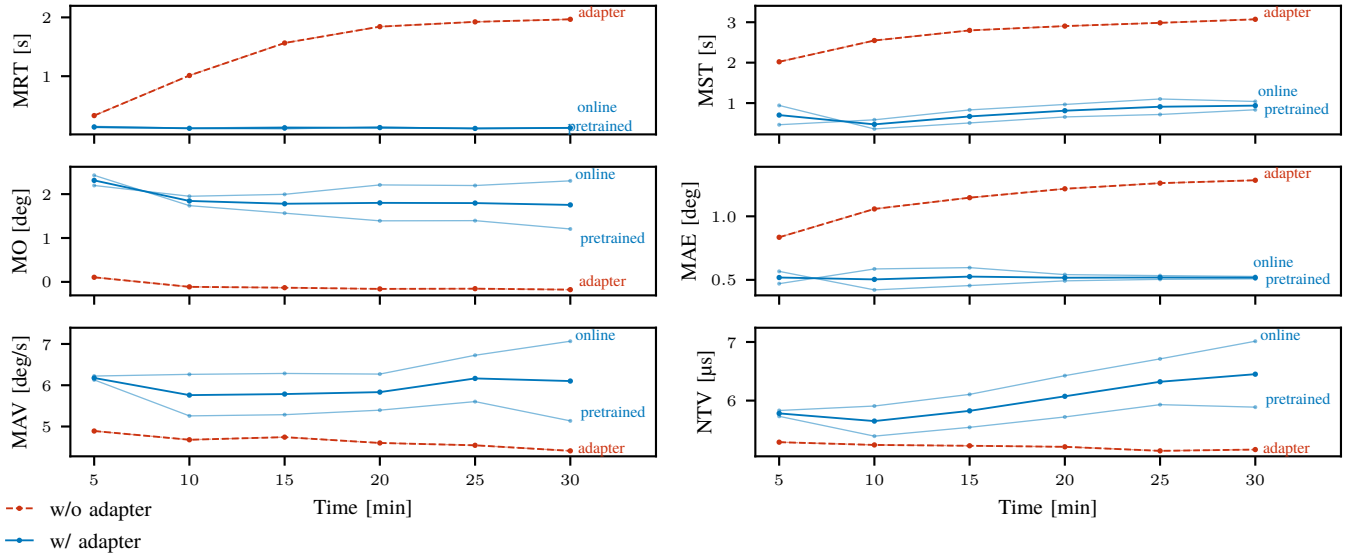


Fig. B.5: Metrics evolution for the crawling gait experiment.

TABLE B.V: Numerical values of the metrics for the crawling gait experiment.

(a) Baseline controller

Metric	5	10	15	20	25	30
MRT [s]	0.33	1.01	1.56	1.84	1.92	1.97
MST [s]	2.02	2.55	2.80	2.90	2.98	3.07
MO [deg]	0.10	-0.11	-0.13	-0.16	-0.15	-0.18
MAE [deg]	0.83	1.06	1.15	1.22	1.26	1.28
MAV [deg/s]	4.89	4.68	4.74	4.60	4.54	4.41
NTV [µs]	5.29	5.24	5.23	5.21	5.14	5.16

(b) Pretrained target adapter

Metric	5	10	15	20	25	30
MRT [s]	0.13	0.11	0.10	0.14	0.11	0.12
MST [s]	0.94	0.36	0.51	0.66	0.72	0.83
MO [deg]	2.43	1.74	1.57	1.39	1.40	1.21
MAE [deg]	0.57	0.42	0.46	0.49	0.50	0.51
MAV [deg/s]	6.13	5.26	5.29	5.40	5.60	5.14
NTV [µs]	5.73	5.39	5.54	5.72	5.93	5.89

(c) Online trained target adapter

Metric	5	10	15	20	25	30
MRT [s]	0.15	0.12	0.14	0.12	0.11	0.12
MST [s]	0.46	0.59	0.83	0.97	1.10	1.04
MO [deg]	2.20	1.95	2.00	2.21	2.20	2.30
MAE [deg]	0.47	0.59	0.60	0.54	0.53	0.53
MAV [deg/s]	6.22	6.26	6.29	6.27	6.73	7.07
NTV [µs]	5.83	5.91	6.11	6.43	6.71	7.01